Team Notebook

[UNI-FIIS] - SaleEnOdeUno

1 Data Structure 2
1.1 2d fenwick tree 2
1.2 centroid decomposition 2
1.3 dynamic segment tree 2
1.4 fenwick tree 2
1.5 hld 2
1.6 hld edge 3
1.7 merge sort tree 3
1.8 multi ordered set 4
1.9 ordered set 4
1.10 persistent segment tree 4
1.11 persistent segment tree lazy 4
1.12 segment tree 5
1.13 segment tree 2d 5
1.14 segment tree beats 5
1.15 segment tree lazy 6
1.16 sparse table 6
1.17 trie bitmask 6
1.18 trie string 7
2 Geometry 7
2.1 1 point 2d 7
2.2 2 segments 2d 7
2.3 3 polygon 2d 7
2.4 4 polarsort 2d 8
2.5 5 convexhull 2d 8
2.6 6 diameter 2d 8
2.7 7 closest pair 2d 8
2.8 8 minkowski sum 2d 8
2.9 convex hull trick 9
2.10 li chao tree 9
3 Graphs 9
3.1 2sat 9
3.2 bellman ford 9
3.3 block cut tree 10
3.4 blossom 10
3.5 dinic 10
3.6 dominator tree 11
3.7 dsu on tree 11
3.8 floyd warshall 11
3.9 kuhn 12
3.10 lca 12
3.11 maxflow mincost 12
3.12 tree bridge 13
4 Math 13
4.1 berlekamp massey 13
4.2 cht 14
4.3 criba 14
4.4 criba lineal 14
4.5 diophantine 14
4.6 dirichlet convolution 15
4.7 fft 15
4.8 fwht bitwise convolution 16
4.9 gauss 16
4.10 geo2d 17
4.11 matrix 17
4.12 millerrabin polardrho 17
4.13 ntt 18
4.14 online fft 18
4.15 primitive root 19
4.16 random 19
4.17 theorems 19
4.17.1 Combinatoria 19
4.17.1.1 General 19
4.17.1.2 Triangulo de Pascal 19
4.17.1.3 Numeros de Catalan 19
4.17.1.4 Numeros de Stirling (Primera y Segunda Especie) 20
4.17.1.5 Numeros de Bell 20
4.17.2 Matematicas 20
4.17.2.1 Formulas Generales 20
4.17.2.2 Formulas de Vieta 20
4.17.2.3 Sucesion de Fibonacci 20
4.17.2.4 Ternas Pitagoricas 20
4.17.3 Teoria de Numeros 20
4.17.3.1 Funcion Totiente de Euler (φ) . 20
4.17.3.2 Funcion e Inversion de Mobius . 20
4.17.3.3 MCD (GCD) y MCM (LCM) . . 20
4.17.4 Operaciones de Bits (Bitwise) y Miscelanea 20
4.17.4.1 Teorema de Erdos-Gallai 20
4.17.5 Geometria 20
4.17.5.1 Teorema de Pick 20
4.18 xor basis 21
5 Strings 21
5.1 aho corasick 21
5.2 hashing 21
5.3 manacher 21
5.4 prefix function 21
5.5 suffix array 22
5.6 z function 22

1 Data Structure

1.1 2d fenwick tree

```
struct BIT2D {
    vector<vector<int>> t;
    int n, m;
    BIT2D(int n, int m) : n(n), m(m) {
        t.assign(n, vector<int>(m, 0));
    }
    void update(int x, int y, int add) {
        for (int i = x; i < n; i |= i + 1)
            for (int j = y; j < m; j |= j + 1) t[i][j] +=
add;
    }
    int sum(int x, int y) {
        int ans = 0;
        for (int i = x; i >= 0; i = (i & (i + 1)) - 1)
            for (int j = y; j >= 0; j = (j & (j + 1)) - 1)
ans += t[i][j];
        return ans;
    }
    int query(int lx, int rx, int ly, int ry) {
        return sum(rx, ry) - sum(lx - 1, ry) - sum(rx, ly -
1) + sum(lx - 1, ly - 1);
    }
};
```

1.2 centroid decomposition

```
struct Centroid_Decomposition{
    vector<int> adj[N];
    int sz[N],vis[N],aux;
    int pa[N];

    int find(int u,int p,int Sz){
        int mx = 0, c = 0; sz[u] = 1;
        for (auto v : adj[u]){
            if (v == p || vis[v]) continue;
            c ^= find(v, u, Sz);
            sz[u] += sz[v];
            mx = max(mx, sz[v]);
        }
        if (max(mx, Sz-sz[u]) <= Sz/2)
            aux = p, c = u;
        return c;
    }

    void build(int u,int p,int Sz){
        aux = -1, u = find(u, -1, Sz);
        if (aux != -1) sz[aux] = Sz-sz[u];
        vis[u] = 1, sz[u] = Sz, pa[u] = p;
        for (auto v : adj[u])
            if (!vis[v]) build(v, u, sz[v]);
    }
} cd; // by NekoRolly
```

1.3 dynamic segment tree

```
struct nod {
    ll t;
    ll lazy;
    nod *l;
    nod *r;
};
static nod nods[ND];
static int id = 0;
nod *crearnodo() {
    nods[id].t = NEUT, nods[id].lazy = NEUT_LAZY, nods[id].l
= nods[id].r = NULL;
    return &nods[id++];
}
ll f(ll a, ll b) {
    return a + b;
}
void aply(ll val, nod *u, ll l, ll r) {
    u->t = (r - l + 1) * val;
    u->lazy = val;
}
struct segmentTree {
    ll n;
    nod *root = NULL;
    segmentTree(ll n) : n(n) {}
    void push(nod *u, ll l, ll r) {
        if (u->lazy == NEUT_LAZY || l == r) return;
        ll m = (l + r) / 2;
        if (!u->l) u->l = crearnodo();
        if (!u->r) u->r = crearnodo();
        aply(u->lazy, u->l, l, m);
        aply(u->lazy, u->r, m + 1, r);
        u->lazy = NEUT_LAZY;
    }
    ll query(ll ql, ll qr, nod *u, ll l, ll r) {
        if (qr < l || r < ql || !u) return NEUT;
        if (ql <= l && r <= qr) return u->t;
        push(u, l, r);
        ll m = (l + r) / 2;
        return f(query(ql, qr, u->l, l, m), query(ql, qr, u-
>r, m + 1, r));
    }
    ll query(ll ql, ll qr) {
        return query(ql, qr, root, 0, n - 1);
    }
    void update(ll ql, ll qr, ll val, nod *&u, ll l, ll r) {
        if (qr < l || r < ql) return;
        if (!u) u = crearnodo();
        if (ql <= l && r <= qr) aply(val, u, l, r);
        else {
            push(u, l, r);
            ll m = (l + r) / 2;
            update(ql, qr, val, u->l, l, m);
            update(ql, qr, val, u->r, m + 1, r);
            u->t = f(u->l ? u->l->t : NEUT, u->r ? u->r->t :
NEUT);
        }
    }
    void update(ll ql, ll qr, ll val) {
        update(ql, qr, val, root, 0, n - 1);
    }
};
```

1.4 fenwick tree

```
struct FenwickTree { // Index 1
    int n, ft[MX + 1];
    void update(int l, int v) {
        for (; l <= n; l += l & -l) ft[l] += val;
    }
    int query(int l) {
        int ans = 0;
        for (; l > 0; l -= l & -l) ans += t[l];
        return ans;
    }
    int query(int l, int r) { return query(r) - query(l -
1); }
}
```

1.5 hld

```
struct Graph {
    vector<vector<ll>> adj;
    vector<ll> padre, prof, heavy, head, pos, base, L, R;
    segmentTree st;
    ll curpos, n;
    ll dfs(ll u, ll p) {
        padre[u] = p;
        ll szu = 1;
        ll mxsz = 0;
        heavy[u] = 0;
        for (ll v : adj[u])
            if (v != p) {
                prof[v] = prof[u] + 1;
                ll szv = dfs(v, u);
                szu += szv;
                if (mxsz < szv) {
                    mxsz = szv;
                    heavy[u] = v;
                }
            }
        return szu;
    }
    void decompose(ll u, ll h, vector<ll> &a) {
        head[u] = h;
        L[u] = curpos;
```

```
        pos[u] = curpos++;
        base[pos[u]] = a[u - 1];
        if (heavy[u] != 0) decompose(heavy[u], h, a);
        for (ll v : adj[u])
            if (v != padre[u] && v != heavy[u]) decompose(v,
v, a);
        R[u] = curpos - 1;
    }
    void pupdate(ll u, ll val) {
        st.pupdate(pos[u], val);
    }
    void update(ll u, ll v, ll val) {
        while (head[u] != head[v]) {
            if (prof[head[u]] > prof[head[v]]) swap(u, v);
            st.update(pos[head[v]], pos[v], val);
            v = padre[head[v]];
        }
        if (prof[u] > prof[v]) swap(u, v);
        st.update(pos[u], pos[v], val);
    }
    ll query(ll u, ll v) {
        ll ans = INF;
        while (head[u] != head[v]) {
            if (prof[head[u]] > prof[head[v]]) swap(u, v);
            ans = f(ans, st.query(pos[head[v]], pos[v]));
            v = padre[head[v]];
        }
        if (prof[u] > prof[v]) swap(u, v);
        ans = f(ans, st.query(pos[u], pos[v]));
        return ans;
    }
    ll subtreequery(ll u) {
        return st.query(L[u], R[u]);
    }
    void subtreeupdate(ll u, ll val) {
        st.update(L[u], R[u], val);
    }
    Graph(vector<ll> &a, vector<pll> &aristas, ll root =
1) : n(a.size()) {
        adj.assign(n + 1, vector<ll>()), padre.assign(n + 1,
0), prof.assign(n + 1, 0);
        heavy.assign(n + 1, 0), head.assign(n + 1, 0),
pos.assign(n + 1, 0);
        L.assign(n + 1, 0), R.assign(n + 1, 0),
base.resize(n);
        curpos = 0;
        for (auto [u, v] : aristas) {
            adj[u].pb(v);
            adj[v].pb(u);
        }
        prof[root] = 0;
        dfs(root, root);
        decompose(root, root, a);
        st = segmentTree(base);
    }
};
```

1.6 hld edge

```
struct Graph {
    vector<vector<pll>> adj;
    vector<ll> padre, wpadre, prof, heavy, head, pos, base,
L, R;
    segmentTree st;
    ll curpos;
    ll n;
    ll dfs(ll u, ll p, ll wp) {
        padre[u] = p;
        wpadre[u] = wp;
        ll szu = 1;
        ll mxsz = 0;
        heavy[u] = 0;
        for (auto [v, w] : adj[u])
            if (v != p) {
                prof[v] = prof[u] + 1;
                ll szv = dfs(v, u, w);
                szu += szv;
                if (mxsz < szv) mxsz = szv, heavy[u] = v;
            }
        return szu;
    }
    void decompose(ll u, ll h) {
        head[u] = h;
        L[u] = curpos;
        pos[u] = curpos++;
        base[pos[u]] = wpadre[u];
        if (heavy[u] != 0) decompose(heavy[u], h);
        for (auto [v, w] : adj[u])
            if (v != padre[u] && v != heavy[u]) decompose(v,
v);
        R[u] = curpos - 1;
    }
    void pupdate(ll u, ll v, ll val) {
        if (u == padre[v]) swap(u, v);
        st.pupdate(pos[u], val);
    }
    void update(ll u, ll v, ll val) {
        while (head[u] != head[v]) {
            if (prof[head[u]] > prof[head[v]]) swap(u, v);
            st.update(pos[head[v]], pos[v], val);
            v = padre[head[v]];
        }
        if (prof[u] > prof[v]) swap(u, v);
        if (u != v) st.update(pos[u] + 1, pos[v], val);
    }
    ll query(ll u, ll v) {
        ll ans = INF;
        while (head[u] != head[v]) {
            if (prof[head[u]] > prof[head[v]]) swap(u, v);
            ans = f(ans, st.query(pos[head[v]], pos[v]));
            v = padre[head[v]];
        }
        if (prof[u] > prof[v]) swap(u, v);
        if (u != v) ans = f(ans, st.query(pos[u] + 1,
pos[v]));
        return ans;
    }
    ll subtreequery(ll u) {
        if (L[u] == R[u]) return 0;
        return st.query(L[u] + 1, R[u]);
    }
    void subtreeupdate(ll u, ll val) {
        if (L[u] == R[u]) return;
        st.update(L[u] + 1, R[u], val);
    }
    Graph(vector<pair<pll, ll>> &aristas, ll root = 1) :
n(aristas.size() + 1) {
        adj.assign(n + 1, vector<pll>());
        padre.assign(n + 1, 0), wpadre.assign(n + 1, 0),
prof.assign(n + 1, 0);
        heavy.assign(n + 1, 0), head.assign(n + 1, 0),
pos.assign(n + 1, 0);
        L.assign(n + 1, 0), R.assign(n + 1, 0),
base.resize(n);
        curpos = 0;
        for (auto [edge, w] : aristas) {
            auto [u, v] = edge;
            adj[u].pb({v, w});
            adj[v].pb({u, w});
        }
        prof[root] = 0;
        dfs(root, root, 0LL);
        decompose(root, root);
        st = segmentTree(base);
    }
};
```

1.7 merge sort tree

```
struct segmentTree {
    vector<vector<ll>> seg;
    ll n, N;
    segmentTree() {}
    segmentTree(vector<ll> &a) : n(a.size()) {
        N = 1;
        while (N < n) N <<= 1;
        seg.assign(2 * N, vector<ll>());
        for (int i = 0; i < n; i++) seg[i + N] = {a[i]};
        for (int i = N - 1; i > 0; i--) {
            auto &l = seg[i << 1], &r = seg[i << 1 | 1];
            seg[i].resize((ll)l.size() + (ll)r.size());
            merge(all(l), all(r), seg[i].begin());
        }
    }
```

```
    // cantidad de elementos mayores a "a" en el intervalo
[ql,qr]
    ll query1(ll ql, ll qr, ll a) {
        ll l = ql + N, r = qr + N;
        ll ans = 0;
        while (l <= r) {
            if (l & 1) {
                auto &v = seg[l++];
                ans += v.end() - upper_bound(all(v), a);
            }
            if (!(r & 1)) {
                auto &v = seg[r--];
                ans += v.end() - upper_bound(all(v), a);
            }
            l >>= 1;
            r >>= 1;
        }
        return ans;
    }
    // cantidad de elementos menores a "a" en el intervalo
[ql,qr]
    ll query2(ll ql, ll qr, ll a) {
        ll l = ql + N, r = qr + N;
        ll ans = 0;
        while (l <= r) {
            if (l & 1) {
                auto &v = seg[l++];
                ans += lower_bound(all(v), a) - v.begin();
            }
            if (!(r & 1)) {
                auto &v = seg[r--];
                ans += lower_bound(all(v), a) - v.begin();
            }
            l >>= 1;
            r >>= 1;
        }
        return ans;
    }
};
```

1.8 multi ordered set

```
// find_by_order(k) -> return iterator to the k-th element
(0-indexed) - O(log n)
// order_of_key(num) -> # of items strictly smaller than num
- O(log n)

// using less_equal cause that lower_bound works as
upper_bound and viceversa
// also .find() does not work and you can only erase by
iterator now
// using for example erase(upper_bound(x))
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;

template <typename T>
using ordered_set =
    tree<T, null_type, less_equal<T>, rb_tree_tag,
tree_order_statistics_node_update>;

ordered_set<int> s;
```

1.9 ordered set

```
// find_by_order(k) -> return iterator to the k-th element
(0-indexed) - O(log n)
// order_of_key(num) -> # of items strictly smaller than num
- O(log n)
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;

template <typename T>
using ordered_set =
    tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;

ordered_set<int> S;
```

1.10 persistent segment tree

```
const ll MX = 2e5 + 5;
const ll Q = 2e5 + 5;
const ll MXT = 2 * MX + 20 * Q;
struct PST {
    ll root[Q + 1], t[MXT], L[MXT], R[MXT];
    ll n, nodos, lastversion = 0;
    PST(vector<ll> &a) : n(a.size()) {
        nodos = 2 * n - 1;
        root[0] = 1;
        build(a, 1, 0, n - 1);
    }
    ll f(ll a, ll b) {
        return a + b;
    }
    void build(vector<ll> &a, ll id, ll l, ll r) {
        if (l == r) t[id] = a[l];
        else {
            ll m = (l + r) / 2;
            ll idl = id + 1;
            ll idr = id + 2 * (m - l + 1);
            L[id] = idl;
            R[id] = idr;
            build(a, idl, l, m);
            build(a, idr, m + 1, r);
            t[id] = f(t[idl], t[idr]);
        }
    }
    ll query(ll ql, ll qr, ll id, ll l, ll r) {
        if (ql <= l && r <= qr) return t[id];
        ll m = (l + r) / 2;
        if (qr <= m) return query(ql, qr, L[id], l, m);
        if (m + 1 <= ql) return query(ql, qr, R[id], m + 1,
r);
        return f(query(ql, qr, L[id], l, m), query(ql, qr,
R[id], m + 1, r));
    }
    ll query(ll ql, ll qr, ll version = -1) {
        if (version == -1) version = lastversion;
        return query(ql, qr, root[version], 0, n - 1);
    }
    void update(ll pos, ll val, ll prev, ll &id, ll l, ll r)
{
        id = ++nodos;
        if (l == r) t[id] = val;
        else {
            ll m = (l + r) / 2;
            L[id] = L[prev];
            R[id] = R[prev];
            if (pos <= m) update(pos, val, L[prev], L[id],
l, m);
            else update(pos, val, R[prev], R[id], m + 1, r);
            t[id] = f(t[L[id]], t[R[id]]);
        }
    }
    void update(ll pos, ll val, ll version = -1) {
        if (version == -1) version = lastversion;
        lastversion++;
        update(pos, val, root[version], root[lastversion],
0, n - 1);
    }
};
```

1.11 persistent segment tree lazy

```
const int MX = 2e5 + 5;
const int Q = 2e5 + 5;
const int LOG = 20;
const int ND = 2 * MX + 6 * Q * LOG;
const ll NEUT = 0;
const ll NEUT_LAZY = -2 * INF;
struct nod {
    ll t, lazy;
    nod *l, *r;
};
nod nods[ND];
int id = 0;
nod *crearnodo(nod *u = NULL) {
    if (u) nods[id].t = u->t, nods[id].lazy = u->lazy,
nods[id].l = u->l, nods[id].r = u->r;
    else nods[id].t = NEUT, nods[id].lazy = NEUT_LAZY,
nods[id].l = nods[id].r = NULL;
    return &nods[id++];
}
ll f(ll a, ll b) {
    return a + b;
```

```
}
void apply(ll val, nod *u, ll l, ll r) {
    u->t += (r - l + 1) * val;
    if (u->lazy == NEUT_LAZY) u->lazy = val;
    else u->lazy += val;
}
struct PST {
    nod *root[MX + 5];
    ll n;
    ll lastversion = -1;
    PST(vector<ll> &a) : n(a.size()) {
        root[++lastversion] = crearnodo();
        build(a, root[0], 0, n - 1);
    }
    void build(vector<ll> &a, nod *&u, ll l, ll r) {
        if (!u) u = crearnodo();
        if (l == r) u->t = a[l];
        else {
            ll m = (l + r) >> 1;
            build(a, u->l, l, m);
            build(a, u->r, m + 1, r);
            u->t = f(u->l->t, u->r->t);
        }
    }
    void push(nod *u, ll l, ll r) {
        if (u->lazy == NEUT_LAZY || l == r) return;
        ll m = (l + r) >> 1;
        u->l = crearnodo(u->l), u->r = crearnodo(u->r);
        apply(u->lazy, u->l, l, m);
        apply(u->lazy, u->r, m + 1, r);
        u->lazy = NEUT_LAZY;
    }
    nod *update(ll ql, ll qr, ll val, nod *prev, ll l, ll r)
{
        if (qr < l || r < ql) return prev;
        nod *u = crearnodo(prev);
        if (ql <= l && r <= qr) apply(val, u, l, r);
        else {
            push(u, l, r);
            ll m = (l + r) >> 1;
            u->l = update(ql, qr, val, u->l, l, m);
            u->r = update(ql, qr, val, u->r, m + 1, r);
            u->t = f(u->l->t, u->r->t);
        }
        return u;
    }
    void update(ll ql, ll qr, ll val, ll version = -1) {
        if (version == -1) version = lastversion;
        root[++lastversion] = update(ql, qr, val,
root[version], 0, n - 1);
    }
    ll query(ll ql, ll qr, nod *u, ll l, ll r) {
        if (!u || qr < l || r < ql) return NEUT;
        if (ql <= l && r <= qr) return u->t;
        push(u, l, r);
        ll m = (l + r) >> 1;
        return f(query(ql, qr, u->l, l, m), query(ql, qr, u-
>r, m + 1, r));
    }
    ll query(ll ql, ll qr, ll version = -1) {
        if (version == -1) version = lastversion;
        return query(ql, qr, root[version], 0, n - 1);
    }
};
```

1.12 segment tree

```
struct IterativeSegmentTree { // Index 0
    int n;
    ll ST[2 * MX];
    const ll NEUT = 0;
    void build(vector<ll> &a) {
        n = a.size();
        for (int i = n; i < (n << 1); i++) ST[i] = a[i - n];
        for (int i = n - 1; i > 0; i--) ST[i] = f(ST[i <<
1], ST[i << 1 | 1]);
    }
    ll f(ll x, ll y) { return max(x, y); }
    void update(int i, ll val) {
        for (ST[i += n] = val; i >>= 1; ) ST[i] = f(ST[i <<
1], ST[i << 1 | 1]);
    }
    ll query(int l, int r) { // [l, r>
        ll ansL = NEUT, ansR = NEUT;
        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
            if (l & 1) ansL = f(ansL, ST[l++]);
            if (r & 1) ansR = f(ST[--r], ansR);
        }
        return f(ansL, ansR);
    }
};
```

1.13 segment tree 2d

```
struct ST2D { //Index 0
    int n, m;
    vector<vector<ll>> t;
    ll f(ll a, ll b) {
        return a + b;
    }
    ST2D(vector<vector<ll>> &a) {
        n = a.size();
        m = a[0].size();
        t.assign(2 * n, vector<ll>(2 * m, 0));
        for (int i = 0; i < n; i++)
            for (int j = 0; j < m; j++) t[i + n][j + m] =
a[i][j];
        for (int i = n; i < 2 * n; i++)
            for (int j = m - 1; j > 0; j--) t[i][j] = f(t[i]
[j << 1], t[i][j << 1 | 1]);
        for (int i = n - 1; i > 0; i--)
            for (int j = 1; j < 2 * m; j++) t[i][j] = f(t[i
<< 1][j], t[i << 1 | 1][j]);
    }
    void update(int x, int y, ll val) {
        x += n, y += m, t[x][y] = val;
        for (int j = y; j > 1; j >>= 1) t[x][j >> 1] =
f(t[x][j], t[x][j ^ 1]);
        for (int i = x >> 1; i > 0; i >>= 1) {
            int j = y;
            t[i][j] = f(t[i << 1][j], t[i << 1 | 1][j]);
            for (; j > 1; j >>= 1) t[i][j >> 1] = f(t[i][j],
t[i][j ^ 1]);
        }
    }
    ll queryY(int i, int l, int r) {
        ll ans = 0;
        for (l += m, r += m + 1; l < r; l >>= 1, r >>= 1) {
            if (l & 1) ans = f(ans, t[i][l++]);
            if (r & 1) ans = f(t[i][--r], ans);
        }
        return ans;
    }
    ll query(int lx, int rx, int ly, int ry) {
        ll ans = 0;
        for (x1 += n, x2 += n + 1; x1 < x2; x1 >>= 1, x2 >>=
1) {
            if (x1 & 1) ans = f(ans, queryY(x1++, y1, y2));
            if (x2 & 1) ans = f(queryY(--x2, y1, y2), ans);
        }
        return ans;
    }
};
```

1.14 segment tree beats

```
struct node {
    ll sum;
    ll max1, max2, cont; // min para chmax
};
node f(node a, node b) { // simetrico con min para chmax
    node ans;
    ans.sum = a.sum + b.sum;
    ans.max1 = max(a.max1, b.max1);
    if (a.max1 == b.max1) ans.max2 = max(a.max2, b.max2),
ans.cont = a.cont + b.cont;
    else if (a.max1 > b.max1) ans.max2 = max(a.max2,
b.max1), ans.cont = a.cont;
    else ans.max2 = max(a.max1, b.max2), ans.cont = b.cont;
    return ans;
}
struct SegmentTree {
    vector<node> t;
    int n;
    SegmentTree(vector<ll> &a) : n(a.size()) {
```

```
        t.resize(2 * n);
        build(a, 1, 0, n - 1);
    }
    void build(vector<ll> &a, int id, int l, int r) {
        if (l == r) t[id] = {a[l], a[l], -INF, 1}; // chmax:
-INF -> INF
        else {
            int m = (l + r) >> 1;
            int idl = id + 1;
            int idr = id + 2 * (m - l + 1);
            build(a, idl, l, m), build(a, idr, m + 1, r);
            t[id] = f(t[idl], t[idr]);
        }
    }
    void apply(ll val, int id) {
        if (t[id].max1 <= val) return; // chmax: (<=) ->
(>=)
        t[id].sum += t[id].cont * (val - t[id].max1);
        t[id].max1 = val;
    }
    void push(int id, int l, int r) {
        if (l == r) return;
        int m = (l + r) >> 1;
        int idl = id + 1;
        int idr = id + 2 * (m - l + 1);
        apply(t[id].max1, idl);
        apply(t[id].max1, idr);
    }
    void update(int ql, int qr, ll val, int id, int l, int
r) {
        if (qr < l || r < ql || t[id].max1 <= val)
return; //chmax: (<=) -> (>=)
        if (ql <= l && r <= qr && t[id].max2 < val)
apply(val, id); // chmax: (<) -> (>)
        else {
            push(id, l, r);
            int m = (l + r) >> 1;
            int idl = id + 1;
            int idr = id + 2 * (m - l + 1);
            update(ql, qr, val, idl, l, m), update(ql, qr,
val, idr, m + 1, r);
            t[id] = f(t[idl], t[idr]);
        }
    }
    ll query(int ql, int qr, int id, int l, int r) {
        if (qr < l || r < ql) return NEUT;
        if (ql <= l && r <= qr) return t[id].sum;
        push(id, l, r);
        int m = (l + r) >> 1;
        int idl = id + 1;
        int idr = id + 2 * (m - l + 1);
        return query(ql, qr, idl, l, m) + query(ql, qr, idr,
m + 1, r);
    }
};
```

1.15 segment tree lazy

```
struct SegmentTree { // id: index 1. Vector a: Index 0
    vector<ll> t, lazy;
    int n;
    SegmentTree(vector<ll> &a) : n(a.size()) {
        t.assign(2 * n, NEUT), lazy.assign(2 * n,
NEUT_LAZY);
        build(a, 1, 0, n - 1);
    }
    ll f(ll a, ll b) {
        return a + b;
    }
    void aply(ll val, int id, int l, int r) {
        t[id] += (r - l + 1) * val;
        lazy[id] += val;
    }
    void build(vector<ll> &a, int id, int l, int r) {
        if (l == r) t[id] = a[l];
        else {
            int m = (l + r) >> 1;
            int idl = id + 1;
            int idr = id + 2 * (m - l + 1);
            build(a, idl, l, m), build(a, idr, m + 1, r);
            t[id] = f(t[idl], t[idr]);
        }
    }
    void push(int id, int l, int r) {
        if (l == r || lazy[id] == NEUT_LAZY) return;
        int m = (l + r) >> 1;
        int idl = id + 1;
        int idr = id + 2 * (m - l + 1);
        aply(lazy[id], idl, l, m);
        aply(lazy[id], idr, m + 1, r);
        lazy[id] = NEUT_LAZY;
    }
    void update(int ql, int qr, ll val, int id, int l, int
r) {
        if (qr < l || r < ql) return;
        if (ql <= l && r <= qr) aply(val, id, l, r);
        else {
            push(id, l, r);
            int m = (l + r) >> 1;
            int idl = id + 1;
            int idr = id + 2 * (m - l + 1);
            update(ql, qr, val, idl, l, m), update(ql, qr,
val, idr, m + 1, r);
            t[id] = f(t[idl], t[idr]);
        }
    }
    ll query(int ql, int qr, int id, int l, int r) {
        if (qr < l || r < ql) return NEUT;
        if (ql <= l && r <= qr) return t[id];
        push(id, l, r);
        int m = (l + r) >> 1;
        int idl = id + 1;
        int idr = id + 2 * (m - l + 1);
        return f(query(ql, qr, idl, l, m), query(ql, qr,
idr, m + 1, r));
    }
};
```

1.16 sparse table

```
const int LG = __lg(MX) + 1;
int st[MX][LG];
void build(vector<int> &A) { // O(|f| n log n)
    int n = A.size();
    for (int i = 0; i < n; i++) st[i][0] = A[i];
    for (int k = 1; k < LG; k++) {
        for (int i = 0; i + (1 << k) <= n; i++) {
            st[i][k] = min(st[i][k - 1], st[i + (1 << (k -
1))][k - 1]);
        }
    }
}
int query(int l, int r) { // O(|f|)
    int lg = __lg(r - l + 1);
    return min(st[l][lg], st[r - (1 << lg) + 1][lg]);
}
```

1.17 trie bitmask

```
struct Trie{
    int t[N*B][2],sz[N*B];
    int cur;

    void insert(int x){
        int u = 0; sz[u]++;
        for (int i=B-1; i>=0; i--){
            int k = x>>i&1;
            if (t[u][k] == 0) t[u][k] = ++cur;
            u = t[u][k], sz[u]++;
        }
    }

    void erase(int x){
        int u = 0; sz[0]--;
        for (int i=B-1; i>=0; i--){
            int k = x>>i&1;
            u = t[u][k], sz[u]--;
        }
    }

    int get_min(int x){ // min xor
        int u = 0, ans = 0;
        for (int i=B-1; i>=0; i--){
            int k = x>>i&1;
            if (t[u][k] != 0 && sz[t[u][k]] > 0) u = t[u]
[k];
```

```
            else u = t[u][k^1], ans ^= 1<<i;
        }
        return ans;
    }

    int get_max(int x){ // max xor
        int u = 0, ans = 0;
        for (int i=B-1; i>=0; i--){
            int k = x>>i&1;
            if (t[u][k^1] != 0 && sz[t[u][k^1]] > 0)
                u = t[u][k^1], ans ^= 1<<i;
            else u = t[u][k];
        }
        return ans;
    }
} tr; // by NekoRolly
```

1.18 trie string

```
struct Trie{
    int t[N][K],sz[N],cur;

    void insert(string &s){
        int u = 0; sz[u]++;
        for (char &c : s){
            int k = c-'a';
            if (t[u][k] == 0) t[u][k] = ++cur;
            u = t[u][k], sz[u]++;
        }
    }

    void erase(string &s){
        int u = 0; sz[u]--;
        for (char &c : s){
            int k = c-'a';
            u = t[u][k], sz[u]--;
        }
    }
} tr; // by NekoRolly
```

2 Geometry

2.1 1 point 2d

```
// Base 2D - Coordenadas enteras. Los productos deben caber en ll.
struct point {
    ll x, y;
    point() : x(0), y(0) {}
    point(ll _x, ll _y) : x(_x), y(_y) {}
    point operator+(const point &p) const {
        return {x + p.x, y + p.y};
    }
    point operator-(const point &p) const {
        return {x - p.x, y - p.y};
    }
    bool operator==(const point &p) const {
        return x == p.x && y == p.y;
    }
    bool operator<(const point &p) const {
        return tie(x, y) < tie(p.x, p.y);
    }
};
// O(1) - Producto punto de dos vectores
ll dot(const point &a, const point &b) {
    return a.x * b.x + a.y * b.y;
}
// O(1) - Producto cruzado de dos vectores
ll cross(const point &a, const point &b) {
    return a.x * b.y - a.y * b.x;
}
// O(1) - Orientacion de a, b, c; positiva si giro CCW
ll cross(const point &a, const point &b, const point &c) {
    return cross(b - a, c - a);
}
// O(1) - Distancia euclidiana al cuadrado
ll dist2(const point &a, const point &b) {
    return dot(a - b, a - b);
}
```

2.2 2 segments 2d

```
// Requiere: point 2d
// O(1) - Verifica si p pertenece al segmento cerrado ab
bool onSegment(const point &a, const point &b, const point &p) {
    if (cross(a, b, p) != 0) return false;
    return min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x) && min(a.y, b.y) <= p.y &&
           p.y <= max(a.y, b.y);
}
// O(1) - Interseccion de segmentos, incluidos extremos y solapes
bool intersect(const point &a, const point &b, const point &c, const point &d) {
    ll x = cross(a, b, c), y = cross(a, b, d);
    ll z = cross(c, d, a), w = cross(c, d, b);
    if (((x > 0 && y < 0) || (x < 0 && y > 0)) && ((z > 0 && w < 0) || (z < 0 && w > 0)))
        return true;
    if (x == 0 && onSegment(a, b, c)) return true;
    if (y == 0 && onSegment(a, b, d)) return true;
    if (z == 0 && onSegment(c, d, a)) return true;
    if (w == 0 && onSegment(c, d, b)) return true;
    return false;
}
// O(1) - 0 fuera, 1 borde, 2 dentro del triangulo abc
int pointInTriangle(point a, point b, point c, point p) {
    if (cross(a, b, c) == 0) {
        return (onSegment(a, b, p) || onSegment(b, c, p) || onSegment(c, a, p)) ? 1 : 0;
    }
    ll x = cross(a, b, p), y = cross(b, c, p);
    ll z = cross(c, a, p);
    bool neg = x < 0 || y < 0 || z < 0;
    bool pos = x > 0 || y > 0 || z > 0;
    if (neg && pos) return 0;
    if (x == 0 || y == 0 || z == 0) return 1;
    return 2;
}
```

2.3 3 polygon 2d

```
// Requiere: point 2d, segments 2d
// O(n) - Doble del area orientada; positiva para CCW
ll area2(const vector<point> &p) {
    ll ans = 0;
    int n = p.size();
    for (int i = 0; i < n; i++) ans += cross(p[i], p[(i + 1) % n]);
    return ans;
}
// O(n) - 0 fuera, 1 borde, 2 dentro de poligono simple
int pointInPolygon(const vector<point> &p, point q) {
    bool in = false;
    int n = p.size();
    for (int i = 0, j = n - 1; i < n; j = i++) {
        if (onSegment(p[i], p[j], q)) return 1;
        if ((p[i].y > q.y) != (p[j].y > q.y)) {
            ll c = cross(p[i], p[j], q);
            if ((c > 0) == (p[j].y > p[i].y)) in = !in;
        }
    }
    return in ? 2 : 0;
}
// O(log n) - 0 fuera, 1 borde, 2 dentro; hull CCW estricto
int pointInConvex(const vector<point> &p, point q) {
    int n = p.size();
    if (n == 0) return 0;
    if (n == 1) return p[0] == q ? 1 : 0;
    if (n == 2) return onSegment(p[0], p[1], q) ? 1 : 0;
    if (p[0] == q) return 1;
    ll a = cross(p[0], p[1], q);
    ll b = cross(p[0], p[n - 1], q);
    if (a < 0 || b > 0) return 0;
    if (a == 0) return onSegment(p[0], p[1], q) ? 1 : 0;
    if (b == 0) return onSegment(p[0], p[n - 1], q) ? 1 : 0;
    int l = 1, r = n - 1;
    while (r - l > 1) {
        int m = (l + r) >> 1;
        if (cross(p[0], p[m], q) >= 0) l = m;
        else r = m;
    }
    ll c = cross(p[l], p[l + 1], q);
```

```
    if (c < 0) return 0;
    if (c == 0) return onSegment(p[l], p[l + 1], q) ? 1 : 0;
    return 2;
}
// O(n) - Verifica si un poligono simple es convexo
bool isConvex(const vector<point> &p) {
    int n = p.size();
    if (n < 3) return false;
    int d = 0;
    for (int i = 0; i < n; i++) {
        ll c = cross(p[i], p[(i + 1) % n], p[(i + 2) % n]);
        int s = (c > 0) - (c < 0);
        if (s == 0) continue;
        if (d != 0 && d != s) return false;
        d = s;
    }
    return d != 0;
}
```

2.4 4 polarsort 2d

```
// Requiere: point 2d; precondicion: o no pertenece a p
// O(n log n) - Ordena CCW por angulo desde el eje X
positivo
void polarSort(vector<point> &p, point o = {0, 0}) {
    auto half = [](point a) { return a.y > 0 || (a.y == 0 &&
a.x >= 0) ? 0 : 1; };
    sort(all(p), [&](point a, point b) {
        a = a - o, b = b - o;
        if (half(a) != half(b)) return half(a) < half(b);
        ll c = cross(a, b);
        if (c != 0) return c > 0;
        return dot(a, a) < dot(b, b);
    });
}
```

2.5 5 convexhull 2d

```
// Requiere: point 2d
// O(n log n) - Andrew: hull (Antihorario) sin colineales
vector<point> convexHull(vector<point> p) {
    sort(all(p));
    p.erase(unique(all(p)), p.end());
    int n = p.size();
    if (n <= 2) return p;
    vector<point> h;
    for (int i = 0; i < n; i++) {
        while (h.size() > 1 && cross(h[h.size() - 2],
h.back(), p[i]) <= 0) h.pop_back();
        h.pb(p[i]);
    }
    int t = h.size();
    for (int i = n - 2; i >= 0; i--) {
        while (h.size() > t && cross(h[h.size() - 2],
h.back(), p[i]) <= 0) h.pop_back();
        h.pb(p[i]);
    }
    h.pop_back();
    return h;
}
```

2.6 6 diameter 2d

```
// Requiere: point 2d; h = convexHull(p)
// O(n) - Maxima distancia al cuadrado, Rotating Calipers
ll diameter2(const vector<point> &h) {
    int n = h.size();
    if (n < 2) return 0;
    if (n == 2) return dist2(h[0], h[1]);
    ll ans = 0;
    int j = 1;
    for (int i = 0; i < n; i++) {
        int ni = (i + 1) % n;
        while (cross(h[i], h[ni], h[(j + 1) % n]) >
cross(h[i], h[ni], h[j])) j = (j + 1) % n;
        ans = max({ans, dist2(h[i], h[j]), dist2(h[ni],
h[j])});
        int nj = (j + 1) % n;
        if (cross(h[i], h[ni], h[nj]) == cross(h[i], h[ni],
h[j]))
            ans = max({ans, dist2(h[i], h[nj]), dist2(h[ni],
h[nj])});
    }
    return ans;
}
```

2.7 7 closest pair 2d

```
// Requiere: point 2d; al menos 2 puntos (si no, -1)
// O(n log n) - Minima distancia al cuadrado, Divide and
Conquer
ll closestPair2(vector<point> p) {
    int n = p.size();
    if (n < 2) return -1;
    sort(all(p));
    vector<point> t(n);
    auto cmp = [](point a, point b) { return tie(a.y, a.x) <
tie(b.y, b.x); };
    auto dfs = [&](auto &&self, int l, int r) -> ll {
        if (r - l <= 3) {
            ll ans = LLONG_MAX;
            for (int i = l; i < r; i++)
                for (int j = i + 1; j < r; j++) ans =
min(ans, dist2(p[i], p[j]));
            sort(p.begin() + l, p.begin() + r, cmp);
            return ans;
        }
        int m = (l + r) >> 1;
        ll x = p[m].x;
        ll ans = min(self(self, l, m), self(self, m, r));
        merge(p.begin() + l, p.begin() + m, p.begin() + m,
p.begin() + r, t.begin(), cmp);
        copy(t.begin(), t.begin() + r - l, p.begin() + l);
        vector<point> v;
        for (int i = l; i < r; i++) {
            ll dx = p[i].x - x;
            if (dx * dx >= ans) continue;
            for (int j = (int)v.size() - 1; j >= 0; j--) {
                ll dy = p[i].y - v[j].y;
                if (dy * dy >= ans) break;
                ans = min(ans, dist2(p[i], v[j]));
            }
            v.pb(p[i]);
        }
        return ans;
    };
    return dfs(dfs, 0, n);
}
```

2.8 8 minkowski sum 2d

```
// Requiere: point 2d; poligonos convexos CCW, n,m >= 3
// O(n + m) - Suma de Minkowski; puede incluir colineales
vector<point> minkowskiSum(vector<point> a, vector<point> b)
{
    if (a.empty() || b.empty()) return {};
    auto reorder = [](vector<point> &p) {
        int k = 0;
        for (int i = 1; i < (int)p.size(); i++)
            if (tie(p[i].y, p[i].x) < tie(p[k].y, p[k].x)) k
= i;
        rotate(p.begin(), p.begin() + k, p.end());
    };
    reorder(a), reorder(b);
    int n = a.size(), m = b.size();
    a.pb(a[0]), b.pb(b[0]);
    int i = 0, j = 0;
    vector<point> ans;
    while (i < n || j < m) {
        ans.pb(a[i] + b[j]);
        if (i == n) {
            j++;
            continue;
        }
        if (j == m) {
            i++;
            continue;
        }
        ll c = cross(a[i + 1] - a[i], b[j + 1] - b[j]);
        if (c >= 0) i++;
        if (c <= 0) j++;
    }
    return ans;
}
```

2.9 convex hull trick

```
struct Line{
    ll m,b;

    ll y(ll x){ return m*x+b;}
};

struct Convex_Hull_Trick{
    vector<Line> vec;
    int l = 0;

    bool check(int i){
        double x1 = double(vec[i-1].b - vec[i+1].b)/
(vec[i+1].m - vec[i-1].m);
        double x2 = double(vec[i].b - vec[i+1].b)/
(vec[i+1].m - vec[i].m);
        return x1 >= x2;
    }

    void add(Line L){
        vec.push_back(L);
        while (vec.size() >= 3 && check(vec.size()-2))
            vec.erase(vec.end()-2);
        l = min(l, int(vec.size()-1));
    }

    ll query(ll x){ // O(1) amortizado
        while (l+1 < vec.size() && vec[l].y(x) >
vec[l+1].y(x)) l++;
        return vec[l].y(x);
    }

    ll find(ll x){ // O(logn)
        int l = 0, r = vec.size();
        while (l+1 < r){
            int m = (l+r)>>1;
            if (vec[m-1].y(x) > vec[m].y(x)) l = m;
            else r = m;
        }
        return vec[l].y(x);
    }
} cht; // by NekoRolly
```

2.10 li chao tree

```
struct Line{
    ll m,b;

    ll y(ll x){ return m*x+b;}
};

struct Li_Chao_Tree{
    Line t[N*4];

    void update(int id,int l,int r,Line L){
        if (l+1 == r || t[id].b == -infll){
            if (L.y(l) > t[id].y(l)) t[id] = L;
            return;
        }
        int m = (l+r)>>1;
        if (t[id].y(m) < L.y(m)) swap(t[id], L);
        if (t[id].m > L.m) update(id<<1, l, m, L);
        else update(id<<1|1, m, r, L);
    }

    ll query(int id,int l,int r,ll x){
        if (l+1 == r) return t[id].y(x);
        int m = (l+r)>>1;
        if (x < m) return max(t[id].y(x), query(id<<1, l, m,
x));
        else return max(t[id].y(x), query(id<<1|1, m, r,
x));
    }
} lct; // by NekoRolly
```

3 Graphs

3.1 2sat

```
struct SAT { // Index_1
    vector<vector<int>> adj, adjt;
    vector<bool> used;
    vector<int> comp;
    int n;
    stack<int> st;
    SAT(int n) : n(n) {
        adj.resize(2 * n + 1), adjt.resize(2 * n + 1);
        used.assign(2 * n + 1, 0), comp.assign(2 * n + 1,
0);
    }
    void add(int p, bool bp, int q, bool bq) { // add p -> q
        p += (!bp) * n;
        q += (!bq) * n;
        adj[p].pb(q);
        adjt[q].pb(p);
    }
    void addor(int p, bool bp, int q, bool bq) { // add p or
q
        add(p, !bp, q, bq);
        add(q, !bq, p, bp);
    }
    void addxor(int p, bool bp, int q, bool bq) { // add p
xor q
        addor(p, bp, q, bq);
        addor(p, !bp, q, !bq);
    }
    void addnxor(int p, bool bp, int q, bool bq) { // add
~(p xor q)
        addor(p, !bp, q, bq);
        addor(p, bp, q, !bq);
    }
    void dfs(int u) {
        used[u] = 1;
        for (int v : adj[u])
            if (!used[v]) dfs(v);
        st.push(u);
    }
    void _dfs(int u, int c) {
        comp[u] = c;
        for (int v : adjt[u])
            if (!comp[v]) _dfs(v, c);
    }
    bool solve(vector<bool> &ans) {
        ans.resize(n + 1);
        for (int u = 1; u <= 2 * n; u++)
            if (!used[u]) dfs(u);
        int c = 0;
        while (!st.empty()) {
            int u = st.top();
            st.pop();
            if (!comp[u]) _dfs(u, ++c);
        }
        for (int p = 1; p <= n; p++) {
            if (comp[p] == comp[p + n]) return false;
            ans[p] = comp[p] > comp[p + n];
        }
        return true;
    }
};
```

3.2 bellman ford

```
// find distance from source node to all nodes.
// supports negative edge weights.
// returns true if a negative cycle is detected.
bool bellmanFord(int s, int n) { // O(VE)
    d.assign(n + 1, INF);
    vector<bool> cycle(n, false);
    bool f = false;
    for (int i = 0; i < n; i++) {
        f = false;
        for (int u = 0; u < n; u++) {
            for (auto [v, w] : adj[u]) {
                if (d[u] != INF && d[u] + w < d[v]) {
                    d[v] = d[u] + w;
                    parent[v] = u;
                    f = true;
                    if (i == n - 1) cycle[v] = true;
                }
            }
        }
        if (!f) return false;
    }
```

```
    return true;
}
```

3.3 block cut tree

```
vector<int> adj[MX], bcc[MX], bct[2 * MX], st;
// bcc[u] = componentes a la que pertenece el nodo u
// bct[u] = adj del BCT
int tin[MX], low[MX], timer;
// low[u] : minimo tin de los backedges entre todos los de
los desendientes de u en el __dfs
int comp[MX]; // representante de u en el BCT
int c;
void __dfs(int u, int p = -1) {
    tin[u] = low[u] = ++timer;
    st.pb(u);
    bool skip = 1; // Si ha  multiedges
    for (int v : adj[u]) {
        if (v == p && skip) {
            skip = 0;
            continue;
        }
        if (tin[v]) low[u] = min(low[u], tin[v]);
        else {
            __dfs(v, u);
            low[u] = min(low[u], low[v]);
            if (tin[u] <= low[v]) {
                c++;
                int x = -1;
                while (x != v) x = st.back(), st.pop_back(),
bcc[x].pb(c);
                bcc[u].pb(c);
            }
        }
    }
}
void buildBCT(int n) {
    for (int i = 1; i <= n; i++)
        if (!tin[i]) {
            __dfs(i);
            st.pop_back();
            if (bcc[i].empty()) bcc[i].pb(++c);
        }
    for (int u = 1; u <= n; u++) {
        if (bcc[u].size() == 1) comp[u] = bcc[u].back();
        else {
            comp[u] = ++c;
            for (int v : bcc[u]) bct[comp[u]].pb(v),
bct[v].pb(comp[u]);
        }
    }
}
```

3.4 blossom

```
struct Graph {
    vector<int> adj[MX];
    int p[MX];
    int match[MX];
    // match[u] is the node match with u
    // or -1 if it does not belong to the matching

    int base[MX]; // base of the flower
    bool used[MX];
    bool blossom[MX];

    void addEdge(int u, int v);

    int lca(int u, int v, int n) { // O(n)
        vector<bool> marked(n);
        while (true) {
            u = base[u]; // blossom compression
            marked[u] = true;
            if (match[u] == -1) break; // we reach the
root : unexposed vertex
            u = p[match[u]];
        }
        while (true) {
            v = base[v];
            if (marked[v]) return v;
            v = p[match[v]];
        }
        return -1;
    }

    void markPath(int u, int b, int children) {
        while (base[u] != b) {
            blossom[base[u]] = blossom[base[match[u]]] =
true;
            p[u] = children, children = match[u], u =
p[match[u]];
        }
    }

    void compress(int u, int v, int n, queue<int> &q) {
        int curBase = lca(u, v, n); // base of the flower
        assert(curBase != -1);
        fill(blossom, blossom + n, false);

        markPath(u, curBase, v);
        markPath(v, curBase, u);
        for (int i = 0; i < n; i++) {
            if (blossom[base[i]]) {
                base[i] = curBase;
                if (!used[i]) used[i] = true, q.push(i);
            }
        }
    }

    int findPath(int root, int n) {
        for (int i = 0; i < n; i++) base[i] = i, used[i] =
false, p[i] = -1;
        used[root] = true;
        queue<int> q;
        q.push(root);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int to : adj[u]) {
                if (base[u] == base[to] || match[u] == to)
continue;
                if (to == root || (match[to] != -1 &&
p[match[to]] != -1))
                    compress(u, to, n, q); // we find a
blossom
                else if (p[to] == -1) {
                    // augmenting path or continue bfs
                    p[to] = u;
                    if (match[to] == -1) return to; // we
find an augmenting path
                    // we continue the bfs
                    to = match[to], used[to] = true,
q.push(to);
                }
            }
        }
        return -1;
    }

    int maxMatching(int n) { // O(n^3)
        fill(match, match + n, -1);
        int ans = 0;
        for (int u = 0; u < n; u++) {
            if (match[u] == -1) {
                int v = findPath(u, n);
                if (v != -1) {
                    ans++;
                    while (v != -1) {
                        int nextVertex = match[p[v]];
                        match[v] = p[v], match[p[v]] = v, v
= nextVertex;
                    }
                }
            }
        }
        return ans;
    }
} G;
```

3.5 dinic

```
// This implementation is for small graphs G(V, E) (|V| <=
3000 aprox)
// where O(|V|^2) memory will fit the memory limit.
using Cap = long long;
```

```cpp
const int MX = 3000;
const Cap INF = 1e18;
struct Graph {
    vector<int> adj[MX];
    Cap cap[MX][MX];
    Cap flow[MX][MX];
    bool added[MX][MX];
    int level[MX];
    int nextEdge[MX];

    void clear(int n) {
        for (int i = 0; i < n; i++) {
            adj[i].clear();
            for (int j = 0; j < n; j++) cap[i][j] = 0,
flow[i][j] = 0, added[i][j] = false;
        }
    }

    void addEdge(int u, int v, Cap w, bool dir) {
        if (!added[u][v]) {
            adj[u].push_back(v);
            adj[v].push_back(u);
            added[u][v] = added[v][u] = true;
        }
        cap[u][v] += w;
        if (!dir) cap[v][u] += w;
    }

    Cap dfs(int u, int t, Cap f) { // O(E)
        if (u == t) return f;
        for (int &i = nextEdge[u]; i < adj[u].size(); i++) {
            int v = adj[u][i];
            Cap cf = cap[u][v] - flow[u][v];
            if (cf == 0 || level[v] != level[u] + 1)
continue;
            Cap ret = dfs(v, t, min(f, cf));
            if (ret > 0) {
                flow[u][v] += ret;
                flow[v][u] -= ret;
                return ret;
            }
        }
        return 0;
    }

    bool bfs(int s, int t, int n) { // O(E)
        fill(level, level + n, -1);
        queue<int> q({s});
        level[s] = 0;
        while (!q.empty()) {
            int u = q.front();
            nextEdge[u] = 0;
            q.pop();
            for (int v : adj[u]) {
                Cap cf = cap[u][v] - flow[u][v];
                if (level[v] == -1 && cf > 0) {
                    level[v] = level[u] + 1;
                    q.push(v);
                }
            }
        }
        return level[t] != -1;
    }

    Cap maxFlow(int s, int t, int n) {
        // General: O(E * V^2), O(E * V + V * |F|)
        // Unit Cap: O(E * min(E^(1/2) , V^(2/3)))
        // Unit Network: O(E * V^(1/2))
        // In unit network, all the edges have unit capacity
        // and for any vertex except s and t either the
        // incoming or outgoing edge is unique.
        Cap ans = 0;
        while (bfs(s, t, n)) { // O(V) iterations
            // after bfs, the graph is a DAG
            while (Cap inc = dfs(s, t, INF)) ans += inc;
        }
        return ans;
    }
} G;
```

3.6 dominator tree

```cpp
//idom[i]=parent of i in dominator tree with root=rt, or -1
if not exists
int
n,rnk[MAXN],pre[MAXN],anc[MAXN],idom[MAXN],semi[MAXN],low[MAXN];
vector<int> g[MAXN],rev[MAXN],dom[MAXN],ord;
void dfspre(int pos){
    rnk[pos]=SZ(ord); ord.pb(pos);
    for(auto x:g[pos]){
        rev[x].pb(pos);
        if(rnk[x]==n) pre[x]=pos,dfspre(x);
    }
}
int eval(int v){
    if(anc[v]<n&&anc[anc[v]]<n){
        int x=eval(anc[v]);
        if(rnk[semi[low[v]]]>rnk[semi[x]]) low[v]=x;
        anc[v]=anc[anc[v]];
    }
    return low[v];
}
void dominators(int rt){
    for0(i,n){
        dom[i].clear(); rev[i].clear();
        rnk[i]=pre[i]=anc[i]=idom[i]=n;
        semi[i]=low[i]=i;
    }
    ord.clear(); dfspre(rt);
    for(int i=SZ(ord)-1;i;i--){
        int w=ord[i];
        for(int v:rev[w]){
            int u=eval(v);
            if(rnk[semi[w]]>rnk[semi[u]])semi[w]=semi[u];
        }
        dom[semi[w]].pb(w); anc[w]=pre[w];
        for(int v:dom[pre[w]]){
            int u=eval(v);
            idom[v]=(rnk[pre[w]]>rnk[semi[u]]?u:pre[w]);
        }
        dom[pre[w]].clear();
    }
    for(int w:ord) if(w!=rt&&idom[w]!=semi[w])
idom[w]=idom[idom[w]];
    fore(i,0,n) if(idom[i]==n)idom[i]=-1;
}
```

3.7 dsu on tree

```cpp
vector<int> adj[N];
int L[N],V[N],sz[N],cur;

void dfs1(int u,int p){
    L[u] = cur, V[cur++] = u, sz[u] = 1;
    for (int v : adj[u]){
        if (v == p) continue;
        dfs1(v, u);
        sz[u] += sz[v];
    }
}
void dfs2(int u,int p,bool flag){
    int big = -1;
    for (int v : adj[u])
        if (v != p && (big == -1 || sz[big] < sz[v])) big =
v;
    for (int v : adj[u])
        if (v != p && v != big) dfs2(v, u, true);
    if (big != -1) dfs2(big, u, false);
    update(a[u], 1); // add
    for (int v : adj[u]){
        if (v == p || v == big) continue;
        for (int i=L[v]; i<L[v]+sz[v]; i++)
            update(a[V[i]], 1); // add
    }
    ans[u] = dif;
    if (!flag) return;
    for (int i=L[u]; i<L[u]+sz[u]; i++)
        update(a[V[i]], -1); // remove
} // by NekoRolly
```

3.8 floyd warshall

```cpp
void Ini(int n) {
    for0(u, n) {
```

```
        for0(v, n) d[u][v] = INF, parent[u][v] = -1;
        d[u][u] = 0;
    }
}

void addEdge(int u, int v, int w) {
    if (w < d[u][v]) {
        d[u][v] = w;
        parent[u][v] = u;
    }
}

void floydWarshall(int n) { // O(V^3)
    for0(k, n) for0(u, n) for0(v, n) {
        if (d[u][k] == INF) continue;
        if (d[k][v] == INF) continue;
        if (d[u][k] + d[k][v] < d[u][v]) {
            d[u][v] = max(-INF, d[u][k] + d[k][v]);
            parent[u][v] = parent[k][v];
        }
    }

    // negative cycles
    for0(u, n) for0(v, n) for0(k, n) {
        if (d[k][k] < 0 && d[u][k] != INF && d[k][v] != INF)
{
            d[u][v] = -INF;
        }
    }
}
```

3.9 kuhn

```
// Maximum matching for bipartite unweighted graphs
struct Graph {
    int match[MX];
    bool vis[MX];
    vector<int> adj[MX];
    void addEdge(int u, int v);

    bool dfs(int u) {
        if (vis[u]) return false;
        vis[u] = true;
        for (int v : adj[u]) {
            if (match[v] == -1 || dfs(match[v])) {
                match[v] = u, match[u] = v;
                return true;
            }
        }
        return false;
    }

    int maximumMatching(int n) { // O(V * E)
        // If the bipartition is (V1, V2) we can iterate
from the side
        // that have less nodes (say is V1) and achieve O(V1
* E)
        fill(match, match + n, -1);
        int ans = 0;
        for (int u = 0; u < n; u++) {
            if (match[u] == -1) {
                fill(vis, vis + n, false);
                ans += dfs(u);
            }
        }
        return ans;
    }
} G;
```

3.10 lca

```
int tIn[MX], tOut[MX];
int anc[MX][LG];
int timer;

void dfs(int u) {
    tIn[u] = timer++;
    for (int j = 1; j < LG; j++) anc[u][j] = anc[anc[u][j -
1]][j - 1];
    for (int v : adj[u]) {
        if (v == anc[u][0]) continue;
        anc[v][0] = u;
        dfs(v);
    }
    tOut[u] = timer++;
}

bool isAnc(int u, int v) { // is u ancestor of v ?
    return tIn[u] <= tIn[v] && tOut[u] >= tOut[v];
}

int lca(int u, int v) { // O(log n)
    if (isAnc(u, v)) return u;
    if (isAnc(v, u)) return v;
    for (int i = LG - 1; i >= 0; i--) {
        if (!isAnc(anc[u][i], v)) u = anc[u][i];
    }
    return anc[u][0];
}
```

3.11 maxflow mincost

```
using Cap = long long;
using Cost = long long;

const int MX = 5000;
const Cap INF_CAP = 1e18;
const Cost INF_COST = 1e18;

struct Edge {
    int to;
    Cap flow, cap;
    Cost cost;
    int rev; // index of the backward edge in the adj list
of to
    Edge(int to, Cap cap, Cost cost, int rev)
        : to(to), flow(0), cap(cap), cost(cost), rev(rev) {}
};

struct Graph {
    vector<Edge> adj[MX];
    int parentEdge[MX];
    Cost pot[MX];
    //|pot[u]| <= the maximum sum of absolute cost in a path
    // which is also bounded by [(V - 1) * C]
    // where C is the maximum value of |cost(e)| for all
edges e
    void clear(int n) {
        for (int i = 0; i < n; i++) {
            adj[i].clear();
            pot[i] = 0;
        }
    }

    void addEdge(int u, int v, Cap w, Cost cost, bool dir) {
        adj[u].push_back(Edge(v, w, cost, adj[v].size()));
        adj[v].push_back(Edge(u, 0, -cost, adj[u].size() -
1));
        if (!dir) addEdge(v, u, w, cost, true);
    }

    void spfa(int s, int n) { // O(E V)
        vector<bool> inQueue(n, false);
        for (int i = 0; i < n; i++) pot[i] = INF_COST;
        queue<int> q;
        pot[s] = 0;
        inQueue[s] = true;
        q.push(s);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            inQueue[u] = false;
            for (Edge e : adj[u]) {
                int v = e.to;
                if (e.cap - e.flow > 0 && pot[u] + e.cost <
pot[v]) {
                    pot[v] = pot[u] + e.cost;
                    if (!inQueue[v]) q.push(v);
                    inQueue[v] = true;
                }
            }
        }
    }

    void pushFlow(int s, int t, Cap inc) {
```

```
        int v = t;
        while (v != s) {
            Edge &backward = adj[v][parentEdge[v]];
            int u = backward.to;
            Edge &forward = adj[u][backward.rev];
            forward.flow += inc;
            backward.flow -= inc;
            v = u;
        }
    }

    pair<Cap, Cost> dijkstra(int s, int t, int n) { // O(E
log V)
        //<flow, cost>
        using Path = pair<Cost, int>; //<weight, node>
        priority_queue<Path, vector<Path>, greater<Path>> q;
        vector<Cost> d(n, INF_COST);
        vector<Cap> residualCap(n, 0);
        d[s] = 0;
        residualCap[s] = INF_CAP;
        q.push(Path(d[s], s));
        while (!q.empty()) {
            auto [dist, u] = q.top();
            q.pop();
            if (dist != d[u]) continue;
            for (Edge e : adj[u]) {
                int v = e.to;
                Cap cf = e.cap - e.flow;
                Cost cost = e.cost + pot[u] - pot[v];
                if (cf > 0 && d[u] + cost < d[v]) {
                    d[v] = d[u] + cost;
                    q.push(Path(d[v], v));
                    residualCap[v] = min(residualCap[u],
cf);
                    parentEdge[v] = e.rev;
                }
            }
        }
        if (d[t] == INF_COST) return {0, 0};
        for (int i = 0; i < n; i++) {
            if (pot[i] != INF_COST) pot[i] += d[i];
        }
        Cap cf = residualCap[t];
        pushFlow(s, t, cf);
        return {cf, pot[t] * cf};
    }

    pair<Cap, Cost> minCostFlow(int s, int t, int n) {
        // O(E log V * maxFlow)
        // maxFlow <= V * U, where U is the maximum capacity
        // Assumption: Initially no negative cycles
        //<maxFlow, minCost>
        spfa(s, n); // not necessary if there is no negative
edges
        pair<Cap, Cost> inc;
        Cap flow = 0;
        Cost cost = 0;
        do {
            inc = dijkstra(s, t, n);
            flow += inc.first;
            cost += inc.second;
        } while (inc.first > 0);
        return {flow, cost};
    }
} G;
```

3.12 tree bridge

```
vector<int> adj[MX];
int tin[MX], low[MX], timer;
vector<pii> bridges;
int component[MX];
vector<int> nods[MX];
vector<array<int, 3>> adjcomp[MX];
void dfs(int u, int p) {
    tin[u] = low[u] = ++timer;
    for (int v : adj[u])
        if (v != p) {
            if (tin[v]) low[u] = min(low[u], tin[v]);
            else dfs(v, u), low[u] = min(low[u], low[v]);
            if (tin[u] < low[v]) bridges.pb({u, v});
        }
}
bool isbridge(int u, int v) {
    if (tin[u] > tin[v]) swap(u, v);
    return tin[u] < low[v];
}
void dfsc(int u, int comp) {
    component[u] = comp;
    nods[comp].pb(u);
    for (int v : adj[u])
        if (!isbridge(u, v) && !component[v]) dfsc(v, comp);
}
int tree_bridge(int n) {
    dfs(1, 1);
    int comp = 0;
    for (int u = 1; u <= n; u++)
        if (!component[u]) dfsc(u, ++comp);
    for (auto [u, v] : bridges) {
        int cu = component[u], cv = component[v];
        adjcomp[cu].pb({cv, u, v});
        adjcomp[cv].pb({cu, u, v});
    }
    return comp;
}
```

4 Math

4.1 berlekamp massey

```
const int MOD = 998244353;
struct mint {
    int v;
    mint(long long x = 0) : v(x % MOD) {
        if (v < 0) v += MOD;
    }
    mint operator+(mint o) const {
        return v + o.v >= MOD ? v + o.v - MOD : v + o.v;
    }
    mint operator-(mint o) const {
        return v - o.v < 0 ? v - o.v + MOD : v - o.v;
    }
    mint operator*(mint o) const {
        return 1LL * v * o.v % MOD;
    }
    mint pow(long long k) const {
        mint res = 1, base = *this;
        while (k) {
            if (k & 1) res = res * base;
            base = base * base;
            k >>= 1;
        }
        return res;
    }
    mint inv() const {
        return pow(MOD - 2);
    }
};

struct BM {
    vector<mint> base, c;
    // O(N^2) - Obtiene la recurrencia
    BM(vector<mint> s) : base(s) {
        int n = s.size(), L = 0, m = 0;
        vector<mint> C(n, 0), B(n, 0), T;
        C[0] = B[0] = 1;
        mint b = 1;
        for (int i = 0; i < n; i++) {
            m++;
            mint d = s[i];
            for (int j = 1; j <= L; j++) d = d + C[j] * s[i
- j];
            if (d.v == 0) continue;
            T = C;
            mint coef = d * b.inv();
            for (int j = m; j < n; j++) C[j] = C[j] - coef *
B[j - m];
            if (2 * L > i) continue;
            L = i + 1 - L, B = T, b = d, m = 0;
        }
        c.assign(C.begin() + 1, C.begin() + L + 1);
        for (mint &x : c) x = mint(0) - x;
    }
```

```
    // O(L^2 log K) - Obtiene el k-Ã©simo tÃ©rmino
    mint get(long long k) {
        int n = c.size();
        if (k < base.size()) return base[k];
        if (n == 0) return 0;
        auto combine = [&](vector<mint> A, vector<mint> B) {
            vector<mint> res(2 * n + 1, 0);
            for (int i = 0; i <= n; i++)
                for (int j = 0; j <= n; j++) res[i + j] =
res[i + j] + A[i] * B[j];
            for (int i = 2 * n; i > n; i--)
                for (int j = 0; j < n; j++) res[i - 1 - j] =
res[i - 1 - j] + res[i] * c[j];
            res.resize(n + 1);
            return res;
        };
        vector<mint> pol(n + 1, 0), e(n + 1, 0);
        pol[0] = 1;
        if (n > 0) e[1] = 1;
        k++;
        while (k) {
            if (k & 1) pol = combine(pol, e);
            e = combine(e, e);
            k >>= 1;
        }
        mint res = 0;
        for (int i = 0; i < n; i++) res = res + pol[i + 1] *
base[i];
        return res;
    }
};
```

4.2 cht

```
struct Line {
    mutable ll a, b, c;

    bool operator<(Line r) const {
        return a < r.a;
    }
    bool operator<(ll x) const {
        return c < x;
    }
};
// dynamically insert `a*x + b` lines and query for maximum
// at any x all operations have complexity O(log N)
struct LineContainer : multiset<Line, less<>> {
    ll div(ll a, ll b) {
        return a / b - ((a ^ b) < 0 && a % b);
    }

    bool isect(iterator x, iterator y) {
        if (y == end()) return x->c = INF, 0;
        if (x->a == y->a) x->c = x->b > y->b ? INF : -INF;
        else x->c = div(y->b - x->b, x->a - y->a);
        return x->c >= y->c;
    }

    void add(ll a, ll b) {
        // a *= -1, b *= -1 // for min
        auto z = insert({a, b, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y =
erase(y));
        while ((y = x) != begin() && (--x)->c >= y->c)
isect(x, erase(y));
    }

    ll query(ll x) {
        if (empty()) return -INF; // INF for min
        auto l = *lower_bound(x);
        return l.a * x + l.b;
        // return -l.a * x - l.b; // for min
    }
};
```

4.3 criba

```
bool criba[MX];
ll primerfactor[MX], phi[MX];
// criba para primerfactor, phi O(nlog(log(n)))
void completar() {
    for (int i = 0; i < MX; i++) {
        criba[i] = 1;
        primerfactor[i] = i;
        phi[i] = i;
    }
    criba[0] = 0;
    criba[1] = 0;
    for (ll i = 2; i < MX; i++) {
        if (criba[i]) {
            phi[i]--;
            for (ll j = 2 * i; j < MX; j += i) {
                criba[j] = 0;
                phi[j] -= phi[j] / i;
                if (primerfactor[j] == j) primerfactor[j] =
i;
            }
        }
    }
}
vector<pll> factorizacioncriba(ll n) { // O(log(n))
    vector<pll> ans;
    while (n > 1) {
        ll f = primerfactor[n];
        ll pot = 0;
        while (primerfactor[n] == f) {
            n /= f;
            pot++;
        }
        ans.pb({f, pot});
    }
    return ans;
}
vector<ll> divisorescriba(ll n) { // O(d(x))
    vector<ll> fact = {1};
    while (n > 1) {
        ll f = primerfactor[n];
        ll sz = fact.size();
        ll divisor = 1;
        while (n % f == 0) {
            divisor *= f;
            n /= f;
            rep(i, sz) fact.pb(fact[i] * divisor);
        }
    }
    return fact;
}
```

4.4 criba lineal

```
int spf[MX];
int mu[MX];
int phi[MX];
vector<int> primes;
void compute() {
    spf[1] = mu[1] = phi[1] = 1;
    for (int i = 2; i < MX; i++) {
        if (spf[i] == 0) {
            spf[i] = i;
            mu[i] = -1;
            phi[i] = i - 1;
            primes.pb(i);
        }
        for (int p : primes) {
            if (p > spf[i] || i * p >= MX) break;
            spf[i * p] = p;
            if (p == spf[i]) {
                mu[i * p] = 0;
                phi[i * p] = phi[i] * p;
            } else {
                mu[i * p] = -mu[i];
                phi[i * p] = phi[i] * (p - 1);
            }
        }
    }
}
```

4.5 diophantine

```
ll extended_gcd(ll a, ll b, ll &x, ll &y) {
    x = 1, y = 0;
    ll x1 = 0, y1 = 1;
    while (b) {
```

```
        ll q = a / b;
        tie(x, x1) = make_pair(x1, x - q * x1);
        tie(y, y1) = make_pair(y1, y - q * y1);
        tie(a, b) = make_pair(b, a - q * b);
    }
    return a < 0 ? (x *= -1, y *= -1, a *= -1, a) : a;
}
bool diophantine(ll a, ll b, ll c, ll &x, ll &y, ll &g) {
    if (a == 0 && b == 0) return c ? 0 : (x = y = 0, 1);
    g = extended_gcd(a, b, x, y);
    return (c % g == 0 ? (x *= c / g, y *= c / g, 1) : 0);
}
ll divdown(ll a, ll b) {
    return a / b - ((a ^ b) < 0 && a % b);
}
ll divup(ll a, ll b) {
    return a / b + ((a ^ b) >= 0 && a % b);
}
vector<pll> allsolution(ll a, ll b, ll c, ll lx, ll rx, ll
ly, ll ry) {
    // x = x0 + k*(b/g)    y = y0 - k*(a/g)
    ll x0, y0, g;
    if (!diophantine(a, b, c, x0, y0, g)) return {};
    vector<pll> ans;
    if (!g) {
        for (ll x = lx; x <= rx; x++)
            for (ll y = ly; y <= ry; y++) ans.pb({x, y});
        return ans;
    }
    if (!a) {
        ll y = c / b;
        if (!(ly <= y && y <= ry)) return {};
        for (ll x = lx; x <= rx; x++) ans.pb({x, y});
        return ans;
    }
    if (!b) {
        ll x = c / a;
        if (!(lx <= x && x <= rx)) return {};
        for (ll y = ly; y <= ry; y++) ans.pb({x, y});
        return ans;
    }
    ll dx = b / g, dy = a / g;
    ll kmin, kmax;
    if (dx > 0) kmin = divup(lx - x0, dx), kmax = divdown(rx
- x0, dx);
    else kmin = divup(rx - x0, dx), kmax = divdown(lx - x0,
dx);
    if (dy > 0) kmin = max(kmin, divup(y0 - ry, dy)), kmax =
min(kmax, divdown(y0 - ly, dy));
    else kmin = max(kmin, divup(y0 - ly, dy)), kmax =
min(kmax, divdown(y0 - ry, dy));
    for (ll k = kmin; k <= kmax; k++) ans.pb({x0 + k * dx,
y0 - k * dy});
    return ans;
}
```

```
bool positive_sol_minf(ll a, ll b, ll c, ll &x, ll &y, ll
cx, ll cy) {
    // f(x,y)= cx*x + cy*y
    ll g;
    if (!diophantine(a, b, c, x, y, g)) return 0;
    ll dx = b / g, dy = a / g;
    ll kmin = -INF, kmax = INF;
    dx > 0 ? kmin = max(kmin, divup(-x, dx)) : kmax =
min(kmax, divdown(-x, dx));
    dy < 0 ? kmin = max(kmin, divup(y, dy)) : kmax =
min(kmax, divdown(y, dy));
    if (kmin > kmax) return 0;
    ll df = cx * dx - cy * dy;
    ll k = df >= 0 ? kmin : kmax; // df<0 para max
    x += dx * k;
    y -= dy * k;
    return 1;
}
```

4.6 dirichlet convolution

```
ll pref[B];
unordered_map<ll, __int128> dp;
vector<int> primes;
void compute() {
    pref[1] = 1;
    for (int i = 2; i < B; i++) {
        if (pref[i] == 0) {
            pref[i] = i - 1;
            primes.pb(i);
        }
        for (int p : primes) {
            if (i * p >= B) break;
            if (i % p == 0) {
                pref[i * p] = pref[i] * p;
                break;
            } else pref[i * p] = pref[i] * (p - 1);
        }
    }
    for (int i = 1; i < B; i++) pref[i] += pref[i - 1];
}
// Formula:S_f(n) = (1 / g(1)) * (S_f*g(n) - sum( S_f(q) *
( S_g(r) - S_g(l-1) ) )
// q = n/l
// r = n/q
// sum : for(ll l = 2, r; l<=n; l = r+1)
// para f=phi, g=1: phi * 1 = ID
__int128 S_f(ll n) {
    if (n < B) return pref[n];
    if (dp.count(n)) return dp[n];
    __int128 ans = (__int128)n * (n + 1) / 2;
    for (ll l = 2, r; l <= n; l = r + 1) {
        ll q = n / l;
        r = n / q;
        ans -= S_f(q) * (r - l + 1);
    }
    return dp[n] = ans;
}
```

4.7 fft

```
typedef complex<double> cd;
vector<vector<cd>> w;
void calc(int log) {
    w.resize(log + 1);
    w[0].resize(1, 1);
    for (int l = 1, len = 2; l <= log; l++, len <<= 1) {
        w[l].resize(len / 2);
        cd wn(cos(2 * PI / len), sin(2 * PI / len));
        for (int j = 0; 2 * j < len; j++)
            w[l][j] = (j & 1 ? w[l - 1][j / 2] * wn : w[l -
1][j / 2]);
    }
}
void fft(vector<cd> &a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int l = 1, len = 2; len <= n; l++, len <<= 1)
        for (int i = 0; i < n; i += len)
            for (int j = 0; 2 * j < len; j++) {
                cd u = a[i + j], v = a[i + j + len / 2] *
w[l][j];
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
            }
    if (invert) {
        reverse(a.begin() + 1, a.end());
        for (int i = 0; i < n; i++) a[i] /= n;
    }
}
vector<ll> polymult(vector<ll> &a, vector<ll> &b) {
    int n = 1, log = 0;
    while (n < a.size() + b.size() - 1) n <<= 1, log++;
    calc(log);
    vector<cd> A(n);
    for (int i = 0; i < n; i++) A[i] = cd(i < a.size() ?
a[i] : 0, i < b.size() ? b[i] : 0);
    fft(A, 0);
    for (int i = 0; i < n; i++) A[i] *= A[i];
    fft(A, 1);
    vector<ll> ans(a.size() + b.size() - 1);
    for (int i = 0; i < ans.size(); i++) ans[i] =
round(A[i].imag() / 2);
    return ans;
}
```

```cpp
// 2D-FFT
void fft2D(vector<vector<cd>> &a, bool invert) {
    int n = a.size(), m = a[0].size();
    for (int i = 0; i < n; i++) fft(a[i], invert);
    vector<cd> b(n);
    for (int j = 0; j < m; j++) {
        for (int i = 0; i < n; i++) b[i] = a[i][j];
        fft(b, invert);
        for (int i = 0; i < n; i++) a[i][j] = b[i];
    }
}
vector<vector<ll>> convolution2D(vector<vector<ll>> &a,
vector<vector<ll>> &b) {
    int n = 1, logn = 0;
    while (n < a.size() + b.size() - 1) n <<= 1, logn++;
    int m = 1, logm = 0;
    while (m < a[0].size() + b[0].size() - 1) m <<= 1, logm+
+;
    calc(max(logn, logm));
    vector<vector<cd>> A(n, vector<cd>(m));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            A[i][j] = cd(i < a.size() && j < a[0].size() ?
a[i][j] : 0,
                         i < b.size() && j < b[0].size() ?
b[i][j] : 0);
    fft2D(A, 0);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) A[i][j] *= A[i][j];
    fft2D(A, 1);
    vector<vector<ll>> ans(a.size() + b.size() - 1,
vector<ll>(a[0].size() + b[0].size() - 1));
    for (int i = 0; i < ans.size(); i++)
        for (int j = 0; j < ans[0].size(); j++) ans[i][j] =
round(A[i][j].imag() / 2);
    return ans;
}
```

4.8 fwht bitwise convolution

```cpp
//OR-CONVOLUTION
void fwht(vector<ll> &a) {
    int n = a.size();
    for (int s = 2, h = 1; s <= n; s <<= 1, h <<= 1)
        for (int i = 0; i < n; i += s)
            for (int j = i; j < i + h; j++) a[j + h] +=
a[j];
}
void ifwht(vector<ll> &A) {
    int n = A.size();
    for (int s = n, h = s >> 1; s >= 2; s >>= 1, h >>= 1)
        for (int i = 0; i < n; i += s)
            for (int j = i; j < i + h; j++) A[j + h] -=
A[j];
}
```

```cpp
vector<ll> OrConvolution(vector<ll> a, vector<ll> b) {
    int n = 1;
    while (n < a.size() || n < b.size()) n <<= 1;
    a.resize(n), b.resize(n);
    fwht(a), fwht(b);
    vector<ll> conv(n);
    for (int i = 0; i < n; i++) conv[i] = a[i] * b[i];
    ifwht(conv);
    return conv;
}
vector<ll> SubsetConvolution(vector<ll> a, vector<ll> b) {
    int n = 1, bit = 1;
    while (n < a.size() || n < b.size()) n <<= 1, bit++;
    a.resize(n), b.resize(n);
    vector<vector<ll>> A(bit + 1, vector<ll>(n)), B(bit + 1,
vector<ll>(n));
    for (int i = 0; i < n; i++) {
        int f = __builtin_popcount(i);
        A[f][i] = a[i], B[f][i] = b[i];
    }
    for (int i = 0; i <= bit; i++) fwht(A[i]), fwht(B[i]);
    vector<ll> conv(n, 0);
    for (int k = 0; k < n; k++) {
        int f = __builtin_popcount(k);
        for (int i = 0; i <= f; i++) conv[k] += A[i][k] *
B[f - i][k];
    }
    ifwht(conv);
    return conv;
}
//AND-CONVOLUTION
void fwht(vector<ll> &a) {
    int n = a.size();
    for (int s = 2, h = 1; s <= n; s <<= 1, h <<= 1)
        for (int i = 0; i < n; i += s)
            for (int j = i; j < i + h; j++) a[j] += a[j +
h];
}
void ifwht(vector<ll> &A) {
    int n = A.size();
    for (int s = n, h = n >> 1; s >= 2; s >>= 1, h >>= 1)
        for (int i = 0; i < n; i += s)
            for (int j = i; j < i + h; j++) A[j] -= A[j +
h];
}
vector<ll> Andconvolution(vector<ll> a, vector<ll> b) {
    int n = 1;
    while (n < a.size() || n < b.size()) n <<= 1;
    a.resize(n), b.resize(n);
    fwht(a), fwht(b);
    vector<ll> conv(n);
    for (int i = 0; i < n; i++) conv[i] = a[i] * b[i];
    ifwht(conv);
    return conv;
}
```

```cpp
//XOR-CONVOLUTION
void fwht(vector<ll> &a) {
    int n = a.size();
    for (int s = 2, h = 1; s <= n; s <<= 1, h <<= 1)
        for (int i = 0; i < n; i += s)
            for (int j = i; j < i + h; j++) {
                ll a1 = a[j + h], a0 = a[j];
                a[j + h] = a0 - a1, a[j] = a0 + a1;
            }
}
void ifwht(vector<ll> &A) {
    fwht(A);
    int n = A.size();
    for (int i = 0; i < n; i++) A[i] /= n;
}
vector<ll> Xorconvolution(vector<ll> a, vector<ll> b) {
    int n = 1;
    while (n < a.size() || n < b.size()) n <<= 1;
    a.resize(n), b.resize(n);
    fwht(a), fwht(b);
    vector<ll> conv(n);
    for (int i = 0; i < n; i++) conv[i] = a[i] * b[i];
    ifwht(conv);
    return conv;
}
```

4.9 gauss

```cpp
const int N = 3e5 + 9;
const double eps = 1e-9;
int Gauss(vector<vector<double>> a, vector<double> &ans) {
    int n = (int)a.size(), m = (int)a[0].size() - 1;
    vector<int> pos(m, -1);
    double det = 1;
    int rank = 0;
    for (int col = 0, row = 0; col < m && row < n; ++col) {
        int mx = row;
        for (int i = row; i < n; i++)
            if (fabs(a[i][col]) > fabs(a[mx][col])) mx = i;
        if (fabs(a[mx][col]) < eps) {
            det = 0;
            continue;
        }
        for (int i = col; i <= m; i++) swap(a[row][i], a[mx]
[i]);
        if (row != mx) det = -det;
        det *= a[row][col];
        pos[col] = row;
        for (int i = 0; i < n; i++) {
            if (i != row && fabs(a[i][col]) > eps) {
                double c = a[i][col] / a[row][col];
                for (int j = col; j <= m; j++) a[i][j] -=
a[row][j] * c;
            }
        }
```

```
        ++row;
        ++rank;
    }
    ans.assign(m, 0);
    for (int i = 0; i < m; i++) {
        if (pos[i] != -1) ans[i] = a[pos[i]][m] / a[pos[i]]
[i];
    }
    for (int i = 0; i < n; i++) {
        double sum = 0;
        for (int j = 0; j < m; j++) sum += ans[j] * a[i][j];
        if (fabs(sum - a[i][m]) > eps) return -1; // no
solution
    }
    for (int i = 0; i < m; i++)
        if (pos[i] == -1) return 2; // infinte solutions
    return 1;                       // unique solution
}
```

4.10 geo2d

```
struct point {
    ll x, y;
    point() {}
    point(ll _x, ll _y) : x(_x), y(_y) {}
    point operator+(const point &p) const {
        return {x + p.x, y + p.y};
    }
    point operator-(const point &p) const {
        return {x - p.x, y - p.y};
    }
    bool operator==(const point &p) const {
        return x == p.x && y == p.y;
    }
    bool operator<(const point &p) const {
        if (x == p.x) return y < p.y;
        return x < p.x;
    }
};
// Producto punto
long long dot(const point &a, const point &b) {
    return a.x * b.x + a.y * b.y;
}
// Producto cruzado (determinante)
long long cross(const point &a, const point &b) {
    return a.x * b.y - a.y * b.x;
}
// Cross entre vectores AB y AC
long long cross(const point &a, const point &b, const point
&c) {
    return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x -
a.x);
}
// El punto p estÃ¡ en el segmento ab?
bool onSegment(const point &a, const point &b, const point
&p) {
    if (cross(a, b, p) != 0) return false;
    return min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x) &&
min(a.y, b.y) <= p.y &&
           p.y <= max(a.y, b.y);
}
// Interseccion de 2 segmentos
bool intersect(const point &p1, const point &q1, const point
&p2, const point &q2) {
    long long d1 = cross(p1, q1, p2);
    long long d2 = cross(p1, q1, q2);
    long long d3 = cross(p2, q2, p1);
    long long d4 = cross(p2, q2, q1);
    if ((d1 > 0 && d2 < 0 || d1 < 0 && d2 > 0) && (d3 > 0 &&
d4 < 0 || d3 < 0 && d4 > 0))
        return true;
    if (d1 == 0 && onSegment(p1, q1, p2)) return true;
    if (d2 == 0 && onSegment(p1, q1, q2)) return true;
    if (d3 == 0 && onSegment(p2, q2, p1)) return true;
    if (d4 == 0 && onSegment(p2, q2, q1)) return true;
    return false;
}
bool pointInPolygon(const vector<point> &poly, const point
&p) {
    bool inside = false;
    int n = poly.size();
    for (int i = 0, j = n - 1; i < n; j = i++) {
        const point &a = poly[i];
        const point &b = poly[j];
        if (onSegment(a, b, p)) return true;
        if ((a.y > p.y) != (b.y > p.y)) {
            long long x_intersection = (b.x - a.x) * (p.y -
a.y) - (b.y - a.y) * (p.x - a.x);
            long long y_diff = b.y - a.y;
            bool right = (x_intersection * y_diff) < 0;
            if (right) inside = !inside;
        }
    }
    return inside;
}
```

4.11 matrix

```
#define matrix array<array<int, N>, N>
matrix multiply(matrix &a, matrix &b) {
    matrix ans{};
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            for (int k = 0; k < N; k++) ans[i][j] =
smod(ans[i][j], mult(a[i][k], b[k][j]));
    return ans;
}
matrix pow(matrix &a, ll e) {
    matrix ans{};
    for (int i = 0; i < N; i++) ans[i][i] = 1;
    while (e) {
        if (e & 1) ans = multiply(ans, a);
        a = multiply(a, a);
        e >>= 1;
    }
    return ans;
}
```

4.12 millerrabin polardrho

```
ll mult(ll a, ll b, ll mod) {
    return (int128)a * b % mod;
}
ll qp(ll a, ll e, ll mod) {
    ll ans = 1;
    while (e) {
        if (e & 1) ans = mult(ans, a, mod);
        a = mult(a, a, mod);
        e >>= 1;
    }
    return ans;
}
bool compositetest(ll a, ll d, ll s, ll n) {
    ll x = qp(a, d, n);
    if (x == 1 || x == n - 1) return 0;
    for (ll i = 0; i < s - 1; i++) {
        x = mult(x, x, n);
        if (x == n - 1) return 0;
    }
    return 1;
}
bool esprimo(ll n) {
    if (n < 2) return 0;
    ll d = n - 1, s = 0;
    while ((d & 1) == 0) d >>= 1, s++;
    for (ll a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31,
37}) {
        if (n == a) return 1;
        if (compositetest(a, d, s, n)) return 0;
    }
    return 1;
}
mt19937_64
rng(chrono::steady_clock::now().time_since_epoch().count());
ll random(ll a, ll b) {
    return uniform_int_distribution<ll>(a, b)(rng);
}
ll f(ll x, ll c, ll mod) {
    ll m = mult(x, x, mod);
    return m + c >= mod ? m + c - mod : m + c;
}
ll getfactor(ll n) {
    if (n % 2 == 0) return 2;
    if (esprimo(n)) return n;
    while (true) {
```

```
        ll y = random(1, n - 1);
        ll c = random(1, n - 1);
        ll m = 64, g = 1, r = 1, q = 1;
        ll x, ys;
        while (g == 1) {
            x = y;
            for (ll i = 0; i < r; i++) y = f(y, c, n);
            ll k = 0;
            while (k < r && g == 1) {
                ys = y;
                ll lim = min(m, r - k);
                for (ll i = 0; i < lim; i++) {
                    y = f(y, c, n);
                    ll diff = x > y ? x - y : y - x;
                    q = mult(q, diff, n);
                }
                g = __gcd(q, n);
                k += m;
            }
            r <<= 1;
        }
        if (g == n) {
            do {
                ys = f(ys, c, n);
                ll diff = x > ys ? x - ys : ys - x;
                g = __gcd(diff, n);
            } while (g == 1);
        }

        if (g > 1 && g < n) return g;
    }
}
vector<pll> fp(ll n) {
    stack<ll> st;
    st.push(n);
    vector<pll> ans;
    map<ll, ll> cont;
    while (!st.empty()) {
        ll x = st.top();
        st.pop();
        if (x == 1) continue;
        if (esprimo(x)) cont[x]++;
        else {
            ll f = getfactor(x);
            st.push(f), st.push(x / f);
        }
    }
    for (auto [x, f] : cont) ans.pb({x, f});
    return ans;
}
```

4.13 ntt

```
const ll MOD = 998244353;
vector<vector<int>> w;
int pow2, root;
int smod(int a, int b, int mod) {return (a + b >= mod ? a + b - mod : a + b);}
int rmod(int a, int b, int mod) {return (a - b < 0 ? a - b + mod : a - b);}
int mult(int a, int b, int mod) {return (ll)a * b % mod;}
int qp(int a, int e, int mod) {
    int ans = 1;
    while (e) {
        if (e & 1) ans = mult(ans, a, mod);
        a = mult(a, a, mod);
        e >>= 1;
    }
    return ans;
}
void calc(int log, int mod) {
    w.resize(log + 1);
    w[0].resize(1, 1);
    for (int l = 1, len = 2; l <= log; l++, len <<= 1) {
        w[l].resize(len / 2);
        int wn = qp(root, pow2 / len, mod);
        for (int j = 0; 2 * j < len; j++) {
            if (j & 1) w[l][j] = mult(w[l - 1][j / 2], wn,
mod);
            else w[l][j] = w[l - 1][j / 2];
        }
    }
}
void findroot(int mod) {
    pow2 = 1;
    int k = mod - 1;
    while (!(k & 1)) pow2 <<= 1, k >>= 1;
    root = 1;
    while (qp(root, pow2, mod) != 1 || qp(root, pow2 / 2,
mod) == 1) root++;
}
void ntt(vector<int> &a, bool invert, int mod) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int l = 1, len = 2; len <= n; l++, len <<= 1)
        for (int i = 0; i < n; i += len)
            for (int j = 0; 2 * j < len; j++) {
                int u = a[i + j], v = mult(a[i + j + len /
2], w[l][j], mod);
                a[i + j] = smod(u, v, mod);
                a[i + j + len / 2] = rmod(u, v, mod);
            }
    if (invert) {
        reverse(a.begin() + 1, a.end());
        int invn = qp(n, mod - 2, mod);
        for (int i = 0; i < n; i++) a[i] = mult(a[i], invn,
mod);
    }
}
vector<int> convolution(vector<int> a, vector<int> b, int
mod = MOD) {
    int n = 1, log = 0, rn = a.size() + b.size() - 1;
    while (n < a.size() + b.size() - 1) n <<= 1, log++;
    findroot(mod);
    calc(log, mod);
    a.resize(n), b.resize(n);
    ntt(a, 0, mod), ntt(b, 0, mod);
    for (int i = 0; i < n; i++) a[i] = mult(a[i], b[i],
mod);
    ntt(a, 1, mod);
    a.resize(rn);
    return a;
}
// 2D-FFT
void ntt2D(vector<vector<int>> &a, bool invert, int mod) {
    int n = a.size(), m = a[0].size();
    for (int i = 0; i < n; i++) ntt(a[i], invert, mod);
    vector<int> b(n);
    for (int j = 0; j < m; j++) {
        for (int i = 0; i < n; i++) b[i] = a[i][j];
        ntt(b, invert, mod);
        for (int i = 0; i < n; i++) a[i][j] = b[i];
    }
}
vector<vector<int>> convolution2D(vector<vector<int>> a,
vector<vector<int>> b,
                                  int mod = MOD) {
    int n = 1, logn = 0, rn = a.size() + b.size() - 1;
    while (n < rn) n <<= 1, logn++;
    int m = 1, logm = 0, rm = a[0].size() + b[0].size() - 1;
    while (m < rm) m <<= 1, logm++;
    findroot(mod);
    calc(max(logn, logm), mod);
    a.resize(n), b.resize(n);
    for (int i = 0; i < n; i++) a[i].resize(m),
b[i].resize(m);
    ntt2D(a, 0, mod), ntt2D(b, 0, mod);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) a[i][j] = mult(a[i][j],
b[i][j], mod);
    ntt2D(a, 1, mod);
    a.resize(rn);
    for (int i = 0; i < rn; i++) a[i].resize(rm);
    return a;
}
```

4.14 online fft

```
// A(DP) X G_STATIC
// EcuaciÃ³n: dp[i] += c[i] * sum_{j} (A[j] * G[i - j -
```

```
off])
void dfs_AG(int l, int r, int offset, vector<int> &dp,
vector<int> &A, const vector<int> &G,
            const vector<int> &c) {
    if (l == r) {
        // Aqui ya esta calculado dp[i] = sum_{j} (A[j] *
G[i - j - off]) y dp[j]
        // para j < i ya esta calculado
        dp[l] = mult(dp[l], c[l]);
        A[l] = mult(l, dp[l]);
        return;
    }
    int mid = (l + r) >> 1;
    dfs_AG(l, mid, offset, dp, A, G, c);
    vector<int> a(A.begin() + l, A.begin() + mid + 1);
    vector<int> b(G.begin(), G.begin() + max(0, r - l -
offset + 1));
    vector<int> conv = convolution(a, b);
    for (int i = max(mid + 1, l + offset); i <= min(r,
(int)conv.size() + l + offset - 1); i++)
        dp[i] = smod(dp[i], conv[i - l - offset]);
    dfs_AG(mid + 1, r, offset, dp, A, G, c);
}

// A(DP) X A(DP)
// EcuaciÃ³n SimÃ©trica: dp[i] = c[i] * sum_{j} (A[j] * A[i
- j - off]) + g[i]
// Ejemplo de dependencia: A[i] = i * dp[i]
void dfs_sim(int l, int r, int offset, vector<int> &dp,
vector<int> &A, const vector<int> &c,
             const vector<int> &g) {
    if (l == r) {
        dp[l] = smod(mult(c[l], dp[l]), g[l]);
        A[l] = mult(l, dp[l]);
        return;
    }
    int mid = (l + r) >> 1;
    dfs_sim(l, mid, offset, dp, A, c, g);
    vector<int> a(A.begin() + l, A.begin() + mid + 1);
    vector<int> b =
        (l == 0) ? a : vector<int>(A.begin(), A.begin() +
max(0, r - l - offset + 1));
    vector<int> conv = convolution(a, b);
    for (int i = max(mid + 1, l + offset); i <= min(r,
(int)conv.size() + l + offset - 1); i++)
        dp[i] = smod(dp[i], mult(conv[i - l - offset], 2 - !
l));
    dfs_sim(mid + 1, r, offset, dp, A, c, g);
}

// A(DP) X B(DP)
// EcuaciÃ³n: dp[i] += c[i] * sum_{j} (A[j] * B[i - j -
off])
// A[i] = i * dp[i]
// B[i] = sum_{k=0}^{i} dp[k] (Suma prefija)
```

```
void dfs_gen(int l, int r, int offset, vector<int> &dp,
vector<int> &A, vector<int> &B,
             const vector<int> &c) {
    if (l == r) {
        dp[l] = mult(dp[l], c[l]); // dp[l] = c[i] * sum_{j}
(A[j] * B[i - j - off])
        A[l] = mult(l, dp[l]);
        B[l] = smod((l > 0) ? B[l - 1] : 0, dp[l]);
        return;
    }
    int mid = (l + r) >> 1;
    dfs_gen(l, mid, offset, dp, A, B, c);
    vector<int> a1(A.begin() + l, A.begin() + mid + 1);
    vector<int> b1(B.begin(), B.begin() + max(0, r - l -
offset + 1));
    vector<int> conv1 = convolution(a1, b1);
    for (int i = max(mid + 1, l + offset); i <= min(r,
(int)conv1.size() + l + offset - 1);
         i++)
        dp[i] = smod(dp[i], conv1[i - l - offset]);
    if (l > 0) {
        vector<int> a2(B.begin() + l, B.begin() + mid + 1);
        vector<int> b2(A.begin(), A.begin() + max(0, r - l -
offset + 1));
        vector<int> conv2 = convolution(a2, b2);
        for (int i = max(mid + 1, l + offset); i <= min(r,
(int)conv2.size() + l + offset - 1);
             i++)
            dp[i] = smod(dp[i], conv2[i - l - offset]);
    }
    dfs_gen(mid + 1, r, offset, dp, A, B, c);
}
```

4.15 primitive root

```
int primitive_root(int p) {
    int n = p - 1; // n=phi(p) -> n=p-1 si p es primo
    vector<int> fp;
    for (int i = 2; i * i <= n; i++)
        if (n % i == 0) {
            fp.pb(i);
            while (n % i == 0) n /= i;
        }
    if (n > 1) fp.pb(n);
    n = p - 1;
    bool ok = 0;
    for (int g = 2; g <= n; g++) {
        bool ok = 1;
        for (int x : fp)
            if (qp(g, n / x, p) == 1) {
                ok = 0;
                break;
            }
        if (ok) return g;
    }
    return 0;
}
```

4.16 random

```
static mt19937 rng(chrono::steady_clock::now()
    .time_since_epoch().count());
#define rnd(a,b) (uniform_int_distribution<ll>(a,b)(rng))
```

4.17 theorems

4.17.1 Combinatoria

4.17.1.1 General

- $\sum_{k=0}^{n} \binom{n-k}{k} = \text{Fib}_{n+1}$
- $\binom{n}{k} = \binom{n}{n-k}$
- $\binom{n}{k} + \binom{n}{k+1} = \binom{n+1}{k+1}$
- $k\binom{n}{k} = n\binom{n-1}{k-1}$
- $\sum_{i=0}^{n} \binom{n}{i} = 2^n$

Identidad de Vandermonde:

$$\sum_{k=0}^{r} \binom{m}{k}\binom{n}{r-k} = \binom{m+n}{r}$$

Identidad del Palo de Hockey (Hockey-Stick):

$$\sum_{i=r}^{n} \binom{i}{r} = \binom{n+1}{r+1}$$

4.17.1.2 Triangulo de Pascal

- En una fila p donde p es un numero primo, todos los terminos en esa fila excepto los unos son multiplos de p.
- **Teorema de Kummer**: Para enteros $n \geq m \geq 0$ y un numero primo p, la mayor potencia de p que divide a $\binom{n}{m}$ es igual al numero de acarreos cuando m se suma a $n-m$ en base p.

4.17.1.3 Numeros de Catalan

- Formula general: $C_n = \frac{1}{n+1}\binom{2n}{n}$

- Recurrencia: $C_0 = 1, C_1 = 1$ y $C_n = \sum_{k=0}^{n-1} C_k C_{n-1-k}$

Representa, entre otras cosas, el numero de secuencias de parentesis balanceadas o triangulaciones de un poligono convexo.

4.17.1.4 Numeros de Stirling (Primera y Segunda Especie)

- **Primera especie** (numero de permutaciones con k ciclos disjuntos): $S(n,k) = (n-1) \cdot S(n-1,k) + S(n-1,k-1)$
- **Segunda especie** (maneras de particionar un conjunto de n objetos en k subconjuntos no vacios): $S(n,k) = k \cdot S(n-1,k) + S(n-1,k-1)$

4.17.1.5 Numeros de Bell

Cuenta el numero total de particiones posibles de un conjunto.

$$B_{n+1} = \sum_{k=0}^{n} \binom{n}{k} \cdot B_k$$

4.17.2 Matematicas

4.17.2.1 Formulas Generales

- $a^k - b^k = (a-b) \cdot \left(a^{k-1}b^0 + a^{k-2}b^1 + ... + a^0 b^{k-1}\right)$
- $|a-b| + |b-c| + |c-a| = 2(\max(a,b,c) - \min(a,b,c))$

Identidad de Lagrange:

$$\left(\sum_{k=1}^{n} a_k^2\right)\left(\sum_{k=1}^{n} b_k^2\right) - \left(\sum_{k=1}^{n} a_k b_k\right)^2 = \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}\left(a_i b_j - a_j b_i\right)^2$$

4.17.2.2 Formulas de Vieta

Para un polinomio de grado n: $p(x) = a_n x^n + a_{n-1}x^{n-1} + ... + a_0$ con raices $r_1, r_2, ..., r_n$:

- $r_1 + r_2 + ... + r_n = -\frac{a_{n-1}}{a_n}$
- $r_1 r_2 ... r_n = (-1)^n \frac{a_0}{a_n}$

4.17.2.3 Sucesion de Fibonacci

- $F_n = \sum_{k=0}^{\lfloor \frac{n-1}{2} \rfloor} \binom{n-k-1}{k}$
- **Formula de Binet:** $F_n = \frac{1}{\sqrt{5}}\left(\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n\right)$
- $\sum_{i=1}^{n} F_i = F_{n+2} - 1$
- $\gcd(F_m, F_n) = F_{\gcd(m,n)}$

4.17.2.4 Ternas Pitagoricas

Formula de Euclides para generar ternas donde $a^2 + b^2 = c^2$: Para $m > n > 0$:

$$a = m^2 - n^2, \quad b = 2mn, \quad c = m^2 + n^2$$

4.17.3 Teoria de Numeros

4.17.3.1 Funcion Totiente de Euler (φ)

- $\varphi(n) = n \prod_{p \mid n}\left(1 - \frac{1}{p}\right)$ donde p son factores primos distintos.
- Si a y b son coprimos, entonces $\varphi(a \cdot b) = \varphi(a) \cdot \varphi(b)$
- $\sum_{d \mid n} \varphi(d) = n$

4.17.3.2 Funcion e Inversion de Mobius

$\mu(n)$ toma valores en $\{-1, 0, 1\}$:

- 1 si es libre de cuadrados con cantidad par de factores primos.
- -1 si es libre de cuadrados con cantidad impar de factores primos.
- 0 si contiene algun factor primo al cuadrado.

Teorema de inversion de Mobius: Si $g(n) = \sum_{d \mid n} f(d)$, entonces:

$$f(n) = \sum_{d \mid n} \mu(d) g\left(\frac{n}{d}\right)$$

4.17.3.3 MCD (GCD) y MCM (LCM)

- $\gcd(a,b) \cdot \operatorname{lcm}(a,b) = |a \cdot b|$
- $\gcd(a, \operatorname{lcm}(b,c)) = \operatorname{lcm}(\gcd(a,b), \gcd(a,c))$
- $\sum_{k=1}^{n} \gcd(k,n) = \sum_{d \mid n} d \cdot \varphi\left(\frac{n}{d}\right)$

4.17.4 Operaciones de Bits (Bitwise) y Miscelanea

- $a + b = (a \oplus b) + 2(a \text{ AND } b)$
- $a + b = (a \mid b) + (a \text{ AND } b)$
- El k-esimo bit esta encendido en x si y solo si $x \bmod 2^k \geq 2^{k-1}$.
- $1 \oplus 2 \oplus 3 \oplus ... \oplus (4k-1) = 0$ para todo $k \geq 0$.

4.17.4.1 Teorema de Erdos-Gallai

Una secuencia descendente de grados $d_1 \geq d_2 \geq ... \geq d_n$ puede representar un grafo simple finito si la suma total de grados es par y:

$$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k)$$

para cada $1 \leq k \leq n$

4.17.5 Geometria

4.17.5.1 Teorema de Pick

Para un poligono simple con vertices de coordenadas enteras:

$$A = I + \frac{B}{2} - 1$$

Donde:

- A: Area del poligono.
- I: Numero de puntos enteros estrictamente interiores.
- B: Numero de puntos enteros sobre el borde.

Puntos enteros interiores:

$$I = A - \frac{B}{2} + 1$$

Puntos enteros en un segmento: Para dos puntos (x_1, y_1) y (x_2, y_2):

$$\text{Puntos} = \gcd(|x_2 - x_1|, |y_2 - y_1|) + 1$$

Puntos enteros en el borde de un poligono: Para n vertices consecutivos, con indices ciclicos:

$$B = \sum_{i=0}^{n-1} \gcd(|x_{i+1} - x_i|, |y_{i+1} - y_i|)$$

4.18 xor basis

```
template <typename T, int LOG>
struct Basis {
    T basis[LOG] = {0};
    int dim = 0;
    bool insert(T x) { // FormarÃ¡ parte del Basis?
        for (int i = LOG - 1; i >= 0; i--)
            if ((x >> i) & 1) {
                if (basis[i]) x ^= basis[i];
                else {
                    basis[i] = x, dim++;
                    return true;
                }
            }
        return false;
    }
    T size() {
        return ((T)1 << dim);
    }
    bool find(T x) { // Â¿x â^^ span{x1,x2...}?
        for (int i = LOG - 1; i >= 0; i--) x = min(x, x ^
basis[i]);
        return x == 0;
    }
    T max_xor(T x = 0) { // max (x^a / a â^^ span{x1,x2...})
        for (int i = LOG - 1; i >= 0; i--) x = max(x, x ^
basis[i]);
        return x;
    }
    T kth(T k) { // The k_smaller of span{x1,x2...}
        if (k < 1 || k > ((T)1 << dim)) return -1;
        T x = 0, cont = ((T)1 << dim);
        for (int i = LOG - 1; i >= 0; i--)
            if (basis[i]) {
                if (k > (cont >> 1)) {
                    if (!((x >> i) & 1)) x ^= basis[i];
                    k -= (cont >> 1);
                } else if ((x >> i) & 1) x ^= basis[i];
                cont >>= 1;
            }
        return x;
    }
}
```

5 Strings

5.1 aho corasick

```
struct Aho_Corasick{
    int t[N][K],cur;
    int pa[N],go[N][K],lnk[N];
    queue<int> qu;

    int insert(string &s){
        int u = 0;
        for (char &c : s){
            int k = c-'a';
            if (t[u][k] == 0)
                t[u][k] = ++cur, pa[cur] = u;
            u = t[u][k];
        }
        return u;
    }

    void bfs(){
        fill(lnk+1, lnk+cur+1, -1);
        for (qu.push(0); !qu.empty(); ){
            int u = qu.front(); qu.pop();
            for (int k=0; k<K; k++){
                int v = t[u][k];
                if (v == 0) go[u][k] = go[lnk[u]][k];
                else{
                    lnk[v] = u == 0 ? 0 : go[lnk[u]][k];
                    go[u][k] = t[u][k];
                    qu.push(v);
                }
            }
        }
    }
} aho; // by NekoRolly
```

5.2 hashing

```
struct Hashing{
    const int mod = 1e9+9;
    const int p = 367, q = 467;
    int ep[N],eq[N],prp[N],prq[N];
    int P(int a,int b){ return 1ll*a*b%mod;}
    int S(int a,int b){ return (a+b)%mod;}

    void build(string &s){
        int n = s.size();
        ep[0] = eq[0] = 1;
        prp[0] = prq[0] = 0;
        for (int i=1; i<=n; i++){
            prp[i] = S(P(prp[i-1], p), s[i-1]);
            prq[i] = S(P(prq[i-1], q), s[i-1]);
            ep[i] = P(ep[i-1], p);
            eq[i] = P(eq[i-1], q);
        }
    }

    ll substr(int i,int len){ i++;
        ll hp = S(prp[i+len-1], mod-P(prp[i-1], ep[len]));
        ll hq = S(prq[i+len-1], mod-P(prq[i-1], eq[len]));
        return hp*mod + hq;
    }
} H1, H2; // by NekoRolly
```

5.3 manacher

```
// d1[i] odd number of palindromes centered at i
// d2[i] even number of palindromes centered at (i, i + 1)
void manacher(string &s, vector<int> &d1, vector<int> &d2) {
    int n = s.size();
    d1 = vector<int>(n);
    d2 = vector<int>(n, 0);
    int l = -1, r = -1;
    for (int i = 0; i < n; i++) {
        d1[i] = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
        while (i - d1[i] >= 0 && i + d1[i] < n && s[i -
d1[i]] == s[i + d1[i]]) {
            d1[i]++;
        }
        if (i + d1[i] - 1 > r) r = i + d1[i] - 1, l = i -
(d1[i] - 1);
    }
    l = -1, r = -1;
    for (int i = 0; i < n - 1; i++) {
        d2[i] = (i >= r) ? 0 : min(d2[l + r - i - 1], r -
i);
        while (i - d2[i] >= 0 && i + d2[i] + 1 < n && s[i -
d2[i]] == s[i + d2[i] + 1]) {
            d2[i]++;
        }
        if (i + d2[i] > r && d2[i] > 0) r = i + d2[i], l = i
- d2[i] + 1;
    }
}
```

5.4 prefix function

```
void prefix_function(string &s,int pf[]){
    int n = s.size(); pf[0] = 0;
    for (int i=1; i<n; i++){
        int j = pf[i-1];
        while (j>0 && s[j] != s[i]) j = pf[j-1];
        pf[i] = j + (s[i] == s[j]);
    }
} // by NekoRolly
```

5.5 suffix array

```
enum Comparison { LESS, EQUAL, GREATER };

Comparison getComparison(int a, int b) {
    if (a < b) return Comparison::LESS;
    if (a > b) return Comparison::GREATER;
    return Comparison::EQUAL;
}

const int ALPH = 256;
struct SuffixArray {
    vector<int> suffixArray;
    vector<int> lcp;
    // lcp[i] = largest common preffix of sa[i] and sa[i + 1] (in sorted list)
    vector<int> pos;

    vector<int> sortCyclic(string &s) { // O(n log n)
        int n = s.size();
        vector<int> p(n), c(n), cnt(max(ALPH, n), 0);
        // p[] : sorted array with the starting index of substrings of length 2^k
        // c[] : equivalence class
        // counting sort
        for (int i = 0; i < n; i++) cnt[s[i]]++;
        for (int i = 1; i < ALPH; i++) cnt[i] += cnt[i - 1];
        for (int i = 0; i < n; i++) p[--cnt[s[i]]] = i;

        c[p[0]] = 0;
        int classes = 1;
        for (int i = 1; i < n; i++) {
            if (s[p[i]] != s[p[i - 1]]) classes++;
            c[p[i]] = classes - 1;
        }
        // radix sort
        vector<int> pNew(n), cNew(n);
        for (int k = 0; (1 << k) < n; k++) {
            // sort pNew by the second substring
            for (int i = 0; i < n; i++) {
                pNew[i] = p[i] - (1 << k);
                if (pNew[i] < 0) pNew[i] += n;
                cnt[i] = 0;
            }
            // counting sort in the first substring
            for (int i = 0; i < n; i++) cnt[c[pNew[i]]]++;
            for (int i = 1; i < classes; i++) cnt[i] += cnt[i - 1];
            for (int i = n - 1; i >= 0; i--) {
                p[--cnt[c[pNew[i]]]] = pNew[i];
            }
            cNew[p[0]] = 0;
            classes = 1;
            for (int i = 1; i < n; i++) {
                int x = p[i] + (1 << k);
                if (x >= n) x -= n;
                int y = p[i - 1] + (1 << k);
                if (y >= n) y -= n;
                pair<int, int> cur = {c[p[i]], c[x]};
                pair<int, int> prev = {c[p[i - 1]], c[y]};
                if (cur != prev) classes++;
                cNew[p[i]] = classes - 1;
            }
            c = cNew;
        }
        return p;
    }

    void buildLCP(string &s) { // O(n)
        int n = s.size();
        lcp = vector<int>(n - 1, 0);
        pos = vector<int>(n);
        for (int i = 0; i < n; i++) pos[suffixArray[i]] = i;
        int k = 0;
        for (int r = 0; r < n; r++) {
            if (pos[r] == 0) continue;
            int l = suffixArray[pos[r] - 1];
            while (s[l + k] == s[r + k]) k++;
            lcp[pos[r] - 1] = k;
            k = max(0, k - 1);
        }
    }

    // Suffix array: contain starting indexes of the all the suffixes (sorted)
    SuffixArray(string &s) { // O(n log n)
        char minChar = 'a' - 1;
        s += minChar;
        suffixArray = sortCyclic(s);
        suffixArray.erase(suffixArray.begin());
        s.pop_back();
        buildLCP(s);
    }

    // return the lcp of the suffixes s[l...n - 1] and s[r...n - 1]
    /*int getLCP(int l, int r) {
        assert(l != r);
        int minPos = min(pos[l], pos[r]);
        int maxPos = max(pos[l], pos[r]);
        return min(lcp[minPos], ..., lcp[maxPos - 1])
    }*/

    /*Comparison compare(int l1, int r1, int l2, int r2) {
        // compare two substrings [l1, r1], [l2, r2].
        int curLCP = (l1 == l2) ? suffixArray.size() : getLCP(l1, l2);
        if (r1 < l1 + curLCP) {
            if (r2 < l2 + curLCP) {
                int sz1 = r1 - l1 + 1;
                int sz2 = r2 - l2 + 1;
                return getComparison(sz1, sz2);
            } else return Comparison::LESS;
        } else if (r2 < l2 + curLCP) return Comparison::GREATER;
        return getComparison(pos[l1], pos[l2]);
    }*/
};
```

5.6 z function

```
void z_function(string &s,int zf[]){
    int n = s.size();
    for (int i=1,l=0,r=0; i<n; i++){
        if (i < r) zf[i] = min(zf[i-l], r-i);
        while (i+zf[i] < n && s[i+zf[i]] == s[zf[i]]) zf[i]++;
        if (i+zf[i] > r) l = i, r = i+zf[i];
    }
} // by NekoRolly
```